

User Manual

for S32K1_S32M24X OCU Driver

Document Number: UM2OCUASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	7
3.1 Requirements	7
3.2 Driver Design Summary	7
3.3 Hardware Resources	8
3.4 Deviations from Requirements	8
3.5 Driver Limitations	9
3.6 Driver usage and configuration tips	9
3.7 Runtime errors	9
3.8 Symbolic Names Disclaimer	9
3.9 Usage in non AUTOSAR context	10
4 Configuration Parameters	12
4.1 Ocu	13
4.1.1 Module Ocu	14
4.1.2 Container OcuConfigurationOfOptionalApis	15
4.1.3 Parameter OcuDeInitApi	15
4.1.4 Parameter OcuGetCounterApi	15
4.1.5 Parameter OcuNotificationSupported	16
4.1.6 Parameter OcuSetAbsoluteThresholdApi	16
4.1.7 Parameter OcuSetPinActionApi	16
4.1.8 Parameter OcuSetPinStateApi	17
4.1.9 Parameter OcuSetRelativeThresholdApi	17
4.1.10 Parameter OcuVersionInfoApi	17
4.1.11 Container OcuConfigSet	18
4.1.12 Parameter OcuCountdirection	18
4.1.13 Container OcuChannel	19
4.1.14 Parameter OcuChannelId	19
4.1.15 Parameter OcuAssignedHardwareChannel	19
4.1.16 Parameter OcuDefaultThreshold	20
4.1.17 Parameter OcuHardwareTriggeredAdc	20
4.1.18 Parameter OcuHardwareTriggeredDMA	21
4.1.19 Parameter OcuMaxCounterValue	21
4.1.20 Parameter OcuNotification	22
4.1.21 Parameter OcuOutputPinUsed	22
4.1.22 Parameter OcuOutputPinDefaultState	23
4.1.23 Parameter OcuOutputPinAction	23
4.1.24 Parameter OcuUseMcuReferencePoint	23
4.1.25 Parameter OcuChannelTickDuration	24
4.1.26 Reference OcuChannelEcucPartitionRef	24
4.1.27 Reference OcuHWSpecificSettingsRef	25
4.1.28 Reference OcuMcuClockReferencePoint	25
4.1.29 Container OcuHWSpecificSettings	26
4.1.30 Parameter OcuHardwareChannelId	26
4.1.31 Parameter OcuHardwareElements	26
4.1.32 Parameter OcuClockSource	27

4.1.33	Parameter	OcuPrescale	27
4.1.34	Parameter	OcuPrescale_Alternate	28
4.1.35	Parameter	OcuDebugMode	28
4.1.36	Container	OcuGeneral	29
4.1.37	Parameter	OcuDevErrorDetect	29
4.1.38	Parameter	OcuEnableDualClockMode	30
4.1.39	Parameter	OcuEnableUserModeSupport	30
4.1.40	Parameter	OcuMultiPartitionEnabled	30
4.1.41	Reference	OcuEcucPartitionRef	31
4.1.42	Reference	OcuKernelEcucPartitionRef	31
4.1.43	Container	OcuHwResourceConfig	32
4.1.44	Parameter	OcuHwResourceId	32
4.1.45	Parameter	OcuIsrEnable	33
4.1.46	Parameter	OcuChannelsUsed	33
4.1.47	Container	CommonPublishedInformation	34
4.1.48	Parameter	ArReleaseMajorVersion	34
4.1.49	Parameter	ArReleaseMinorVersion	34
4.1.50	Parameter	ArReleaseRevisionVersion	35
4.1.51	Parameter	ModuleId	35
4.1.52	Parameter	SwMajorVersion	36
4.1.53	Parameter	SwMinorVersion	36
4.1.54	Parameter	SwPatchVersion	36
4.1.55	Parameter	VendorApiInfix	37
4.1.56	Parameter	VendorId	37
4.2	FTM_OCU		38
4.2.1	Module	Ftm_Ocu	39
4.2.2	Parameter	OcuDownCountingSupport	40
5	Module Index		41
5.1	Software Specification		41
6	Module Documentation		42
6.1	Ocu IPL		42
6.1.1	Detailed Description		42
6.1.2	Data Structure Documentation		45
6.1.2.1	struct	Ftm_Ocu_Ip_ChStateType	45
6.1.2.2	struct	Ftm_Ocu_Ip_ChannelConfigType	45
6.1.2.3	struct	Ftm_Ocu_Ip_SpecificIpConfigType	45
6.1.2.4	struct	Ftm_Ocu_Ip_ModuleConfigType	46
6.1.3	Macro Definition Documentation		46
6.1.3.1	FTM_OCU_IP_COMBINE_CH_CTRL_WIDTH		46
6.1.3.2	FTM_OCU_IP_INVALID_CHANNEL_NUM		46
6.1.3.3	FTM_OCU_IP_OUTPUTDISABLED		46
6.1.3.4	FTM_OCU_IP_BDM_MODE_00		47
6.1.3.5	FTM_OCU_IP_BDM_MODE_01		47
6.1.3.6	FTM_OCU_IP_BDM_MODE_10		47
6.1.3.7	FTM_OCU_IP_BDM_MODE_11		47
6.1.3.8	FTM_OCU_IP_CLOCK_SOURCE_NONE		47
6.1.3.9	FTM_OCU_IP_CLOCK_SOURCE_SYSTEMCLK		48
6.1.3.10	FTM_OCU_IP_CLOCK_SOURCE_FIXEDCLK		48
6.1.3.11	FTM_OCU_IP_CLOCK_SOURCE_EXTERNALCLK		48
6.1.3.12	FTM_OCU_IP_CLOCK_DIVID_BY_1		48
6.1.3.13	FTM_OCU_IP_CLOCK_DIVID_BY_2		48
6.1.3.14	FTM_OCU_IP_CLOCK_DIVID_BY_4		49

6.1.3.15	FTM_OCUC_IP_CLOCK_DIVID_BY_8	49
6.1.3.16	FTM_OCUC_IP_CLOCK_DIVID_BY_16	49
6.1.3.17	FTM_OCUC_IP_CLOCK_DIVID_BY_32	49
6.1.3.18	FTM_OCUC_IP_CLOCK_DIVID_BY_64	49
6.1.3.19	FTM_OCUC_IP_CLOCK_DIVID_BY_128	49
6.1.3.20	FTM_OCUC_IP_SET_HIGH	50
6.1.3.21	FTM_OCUC_IP_SET_LOW	50
6.1.3.22	FTM_OCUC_IP_SET_TOGGLE	50
6.1.3.23	FTM_OCUC_IP_SET_DISABLE	50
6.1.3.24	FTM_OCUC_IP_HIGH	50
6.1.3.25	FTM_OCUC_IP_LOW	50
6.1.3.26	FTM_OCUC_IP_PRIMARY_PRESCALER	51
6.1.3.27	FTM_OCUC_IP_ALTERNATIVE_PRESCALER	51
6.1.3.28	FTM_OCUC_IP_CM_IN_REF_INTERVAL	51
6.1.3.29	FTM_OCUC_IP_CM_OUT_REF_INTERVAL	51
6.1.4	Types Reference	51
6.1.4.1	Ftm_Ocu_Ip_PinActionType	51
6.1.4.2	Ftm_Ocu_Ip_PinStateType	52
6.1.4.3	Ftm_Ocu_Ip_SelectPrescalerType	52
6.1.4.4	Ftm_Ocu_Ip_ReturnType	52
6.1.4.5	Ftm_Ocu_Ip_xRegLayoutType	52
6.1.4.6	Ftm_Ocu_Ip_CallbackType	52
6.1.4.7	Ftm_Ocu_Ip_ValueType	52
6.1.5	Enum Reference	53
6.1.5.1	Ftm_Ocu_Ip_OutputCompareModeType	53
6.1.6	Function Reference	53
6.1.6.1	Ftm_Ocu_Ip_Init()	53
6.1.6.2	Ftm_Ocu_Ip_DeInit()	53
6.1.6.3	Ftm_Ocu_Ip_StartChannel()	53
6.1.6.4	Ftm_Ocu_Ip_StopChannel()	54
6.1.6.5	Ftm_Ocu_Ip_GetCounter()	54
6.1.6.6	Ftm_Ocu_Ip_SetPinAction()	54
6.1.6.7	Ftm_Ocu_Ip_SetPinState()	55
6.1.6.8	Ftm_Ocu_Ip_SetRelativeThreshold()	55
6.1.6.9	Ftm_Ocu_Ip_SetAbsoluteThreshold()	55
6.1.6.10	Ftm_Ocu_Ip_SetClockMode()	56
6.1.6.11	Ftm_Ocu_Ip_GetMaxCounterValue()	56
6.1.6.12	Ftm_Ocu_Ip_DisableNotification()	56
6.1.6.13	Ftm_Ocu_Ip_EnableNotification()	57
6.1.6.14	FTM_0_CH_0_CH_1_ISR()	57
6.1.6.15	FTM_0_CH_2_CH_3_ISR()	57
6.1.6.16	FTM_0_CH_4_CH_5_ISR()	57
6.1.6.17	Ftm_Ocu_Ip_SetOutputPinCtrlBit()	58
6.2	Ocu Driver	59
6.2.1	Detailed Description	59
6.2.2	Data Structure Documentation	62
6.2.2.1	struct Ocu_ChannelConfigType	62
6.2.2.2	struct Ocu_ConfigType	62
6.2.3	Macro Definition Documentation	63
6.2.3.1	OCUC_VALID_CHANNEL_NUM	63
6.2.3.2	OCUC_INVALID_CHANNEL_NUM	64
6.2.3.3	OCUC_INSTANCE_INDEX	64
6.2.3.4	OCUC_E_UNINIT	64
6.2.3.5	OCUC_E_PARAM_INVALID_CHANNEL	64

6.2.3.6	OCU_E_PARAM_INVALID_STATE	64
6.2.3.7	OCU_E_PARAM_INVALID_ACTION	64
6.2.3.8	OCU_E_NO_VALID_NOTIF	65
6.2.3.9	OCU_E_ALREADY_INITIALIZED	65
6.2.3.10	OCU_E_PARAM_POINTER	65
6.2.3.11	OCU_E_BUSY	65
6.2.3.12	OCU_E_PARAM_NO_PIN	65
6.2.3.13	OCU_E_INIT_FAILED	66
6.2.3.14	OCU_E_PARAM_INVALID_VALUE	66
6.2.3.15	OCU_E_PARAM_CONFIG	66
6.2.3.16	OCU_INIT_ID	66
6.2.3.17	OCU_DEINIT_ID	66
6.2.3.18	OCU_STARTCHANNEL_ID	67
6.2.3.19	OCU_STOPCHANNEL_ID	67
6.2.3.20	OCU_SETPINSTATE_ID	67
6.2.3.21	OCU_SETPINACTION_ID	67
6.2.3.22	OCU_GETCOUNTER_ID	67
6.2.3.23	OCU_SETABSOLUTETHRESHOLD_ID	68
6.2.3.24	OCU_SETRELATIVETHRESHOLD_ID	68
6.2.3.25	OCU_GETVERSIONINFO_ID	68
6.2.3.26	OCU_DISABLENOTIFICATION_ID	68
6.2.3.27	OCU_ENABLENOTIFICATION_ID	68
6.2.3.28	OCU_SETCLOCKMODE_ID	69
6.2.4	Types Reference	69
6.2.4.1	Ocu_NotificationType	69
6.2.4.2	Ocu_ValueType	69
6.2.4.3	Ocu_ChannelType	69
6.2.5	Enum Reference	69
6.2.5.1	Ocu_ReturnType	69
6.2.5.2	Ocu_PinActionType	70
6.2.5.3	Ocu_PinStateType	70
6.2.5.4	Ocu_SelectPrescalerType	70
6.2.6	Function Reference	71
6.2.6.1	Ocu_Init()	71
6.2.6.2	Ocu_DeInit()	71
6.2.6.3	Ocu_StartChannel()	72
6.2.6.4	Ocu_StopChannel()	72
6.2.6.5	Ocu_SetPinState()	73
6.2.6.6	Ocu_SetPinAction()	74
6.2.6.7	Ocu_GetCounter()	74
6.2.6.8	Ocu_SetAbsoluteThreshold()	75
6.2.6.9	Ocu_SetRelativeThreshold()	76
6.2.6.10	Ocu_DisableNotification()	76
6.2.6.11	Ocu_EnableNotification()	77
6.2.6.12	Ocu_GetVersionInfo()	77
6.2.6.13	Ocu_SetClockMode()	78
6.2.6.14	Ocu_ProcessChannelNotification()	78
6.2.7	Variable Documentation	78
6.2.7.1	Ocu_Config_VS_0	78

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor AUTOSAR Ocu for S32K1_S32M24X. AUTOSAR Ocu driver configuration parameters and deviations from the specification are described in Driver chapter of this document. AUTOSAR Ocu driver requirements and APIs are described in the AUTOSAR Ocu driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k116_qfn32
- s32k116_lqfp48
- s32k118_lqfp48
- s32k118_lqfp64
- s32k142_lqfp48
- s32k142_lqfp64
- s32k142_lqfp100
- s32k142w_lqfp48
- s32k142w_lqfp64
- s32k144_lqfp48
- s32k144_lqfp64 / MWCT1014S_lqfp64
- s32k144_lqfp100 / MWCT1014S_lqfp100
- s32k144_mapbga100
- s32k144w_lqfp48
- s32k144w_lqfp64

- s32k146_lqfp64
- s32k146_lqfp100 / MWCT1015S_lqfp100
- s32k146_mapbga100 / MWCT1015S_mapbga100
- s32k146_lqfp144
- s32k148_lqfp100
- s32k148_mapbga100 / MWCT1016S_mapbga100
- s32k148_lqfp144
- s32k148_lqfp176
- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
AUTOSAR	Automotive Open System Architecture
API	Application Programming Interface
ARTD	Automotive Real Time Drivers
ASR	AUTOSAR
BSW	Basic Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DIO	Digital Input Output
DMA	Direct Memory Access
ECU	Electronic Control Unit
ECUC	ECU Configuration
EcuM	ECU state Manager
eMIOS	Enhanced Modular IO Subsystem
FTM	FlexTimer
GUI	Graphical User Interface
HLD	High Level Driver
HW	Hardware
ICU	Input Capture Unit
IP	Intellectual Property, referred as a hardware design block
IPL	IP Layer
IPW	IP Wrapper Layer
ISR	Interrupt Service Routine
MCAL	Microcontroller Abstraction Layer
MCU	Micro Controller Unit
N/A	Not Applicable
OCU	Output Compare Unit
OS	Operating System
OSIF	OS Interface
PB Variant	Post Build Variant
PC Variant	Pre Compile Variant
PWM	Pulse Width Modulation
RAM	Random Access Memory
ROM	Read-only Memory
SCI	Serial Communication Interface
SoC	System on Chip
SPI	Serial Peripheral Interface
SWS	Software Specification
VLE	Variable Length Encoding
VSMD	Vendor Specific Module Definition
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of Driver	AUTOSAR Release R21-11
2	S32K1xx Reference Manual	Rev.14.1, 01/2024
3	S32M24x Reference Manual	Rev. 5, 12/2024
4	S32K1xx Data Sheet	Rev. 14, 08/2021
5	S32M2xx Data Sheet	Rev. 7, 12/2024
6	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
7	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
8	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
9	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
10	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
11	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
12	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
13	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
14	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)
- [Usage in non AUTOSAR context](#)

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See [Table Reference List](#)).

For CDD: `_driver>` is a Complex Device Driver (CDD), so there are no AUTOSAR requirements regarding this module.

It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

This driver provides services for initializing and controlling the Output Compare Unit module. Ocu module is used to run a counter until a predefined value is reached. When a compare-match occurs, a hardware interrupt and/or a state change of an output pin will be triggered. Ocu configuration consists of logic channels exclusively bound to one of the available hardware channels. The Ocu logic channel contains definitions for the initial value of the counter, the maximum counter value (threshold), a notification callback to a user defined function and the behavior of the hardware channel designated pin. A clock source and a prescaler value (optionally a second prescaler) must be set for the hardware channel.

The Ocu driver assures reentrancy (single partition execution) for the APIs based on the following assumptions:

- the "called-again" API is for a different resource (hardware/logic channel);
- common variables/registers accessed with "rmw" are guarded by Exclusive Areas which need to be correctly implemented in RTE on user side;

3.3 Hardware Resources

#	Hardware IP	Description
1	FTM	FlexTimer

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR OCU Driver software specification in some places. The table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, not available, not testable or out of scope for the OCU Driver.

Term	Definition
N/A	Not available
N/T	Not testable
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, not available, not testable or out of scope for the driver.

Requirement	Status	Description	Notes
SWS_Ocu_00020	N/S	The detection of production errors cannot be switched off.	DEM errors are not used
SWS_Ocu_00023	N/S	Each variable that shall be accessible by AUTOSAR Debugging, shall be defined as global variable.	AUTOSAR Debugging concept is not implemented. In par with PR-MCAL-3330
SWS_Ocu_00024	N/S	All type definitions of variables which shall be debugged shall be accessible by the header file Ocu.h .	AUTOSAR Debugging concept is not implemented. In par with PR-MCAL-3330
SWS_Ocu_00025	N/S	The declaration of variables in the header file shall be such that it is possible to calculate the size of the variables by C-"sizeof".	AUTOSAR Debugging concept is not implemented. In par with PR-MCAL-3330
SWS_Ocu_00026	N/S	Variables available for debugging shall be described in the respective OCU driver Description.	AUTOSAR Debugging concept is not implemented. In par with PR-MCAL-3330
SWS_Ocu_00125	N/S	If source code for caller and callee of <code>Ocu_GetVersionInfo</code> is available; the OCU driver should realize <code>Ocu_GetVersionInfo</code> as a macro, defined in the module's header file.	<code>Ocu_GetVersionInfo</code> is defined as a function as a common approach in all MCAL drivers.
SWS_Ocu_00156	N/S	These requirements are not applicable to this specification	Not a requirement

Requirement	Status	Description	Notes
ECUC_Ocu_00168	N/S	Maps the OCU kernel to zero or one ECUC partitions to assign the driver kernel to a certain core. The ECUC partition referenced is a subset of the ECUC partitions where the OCU driver is mapped to. Tags: atp. Status=draft	Type IV Autosar multicore not implemented for current module (AAI-445), therefore Ocu↔KernelEcucPartitionRef is not supported
SWS_Ocu_CONSTR_00001	N/S	The ECUC partitions referenced by OcuKernelEcucPartitionRef shall be a subset of the ECUC partitions referenced by Ocu↔EcucPartitionRef.	Type IV Autosar multicore not implemented for current module (AAI-445), therefore Ocu↔KernelEcucPartitionRef is not supported
SWS_Ocu_CONSTR_00004	N/S	If OcuEcucPartitionRef references one or more ECUC partitions, OcuKernelEcuc↔PartitionRef shall have a multiplicity of one and reference one of these ECUC partitions as well.	Type IV Autosar multicore not implemented for current module (AAI-445), therefore Ocu↔KernelEcucPartitionRef is not supported.
SWS_Ocu_CONSTR_00005	N/S	If OcuEcucPartitionRef references one or more ECUC partitions, OcuKernelEcuc↔PartitionRef shall have a multiplicity of one and reference one of these ECUC partitions as well	Type IV Autosar multicore not implemented for current module (AAI-445), therefore Ocu↔KernelEcucPartitionRef is not supported

3.5 Driver Limitations

None.

3.6 Driver usage and configuration tips

In this chapter, the extra features from our drivers that are not described in the AutoSAR standard are detailed.

3.7 Runtime errors

None.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: `#define <Mip>Conf_<Container_ShortName>_<Container_ID>`

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

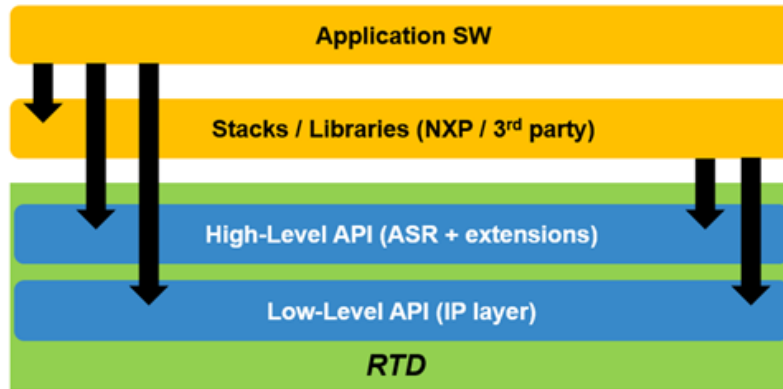


Figure 3.1 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
 1. Initialization of the device clock tree so other periphery can be initialized
 2. Configuration of the pinout so peripherals with external connections can communicate with the outside world
 3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
 4. Initialization of specific drivers
 5. Runtime functionality
 6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

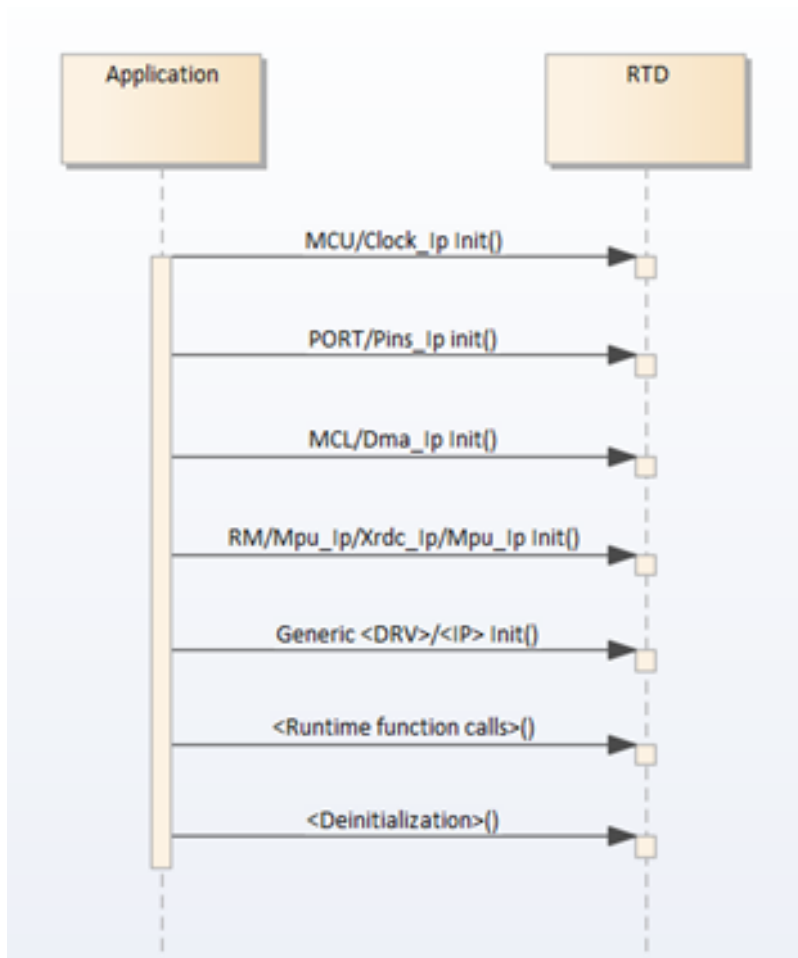


Figure 3.2 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**.xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

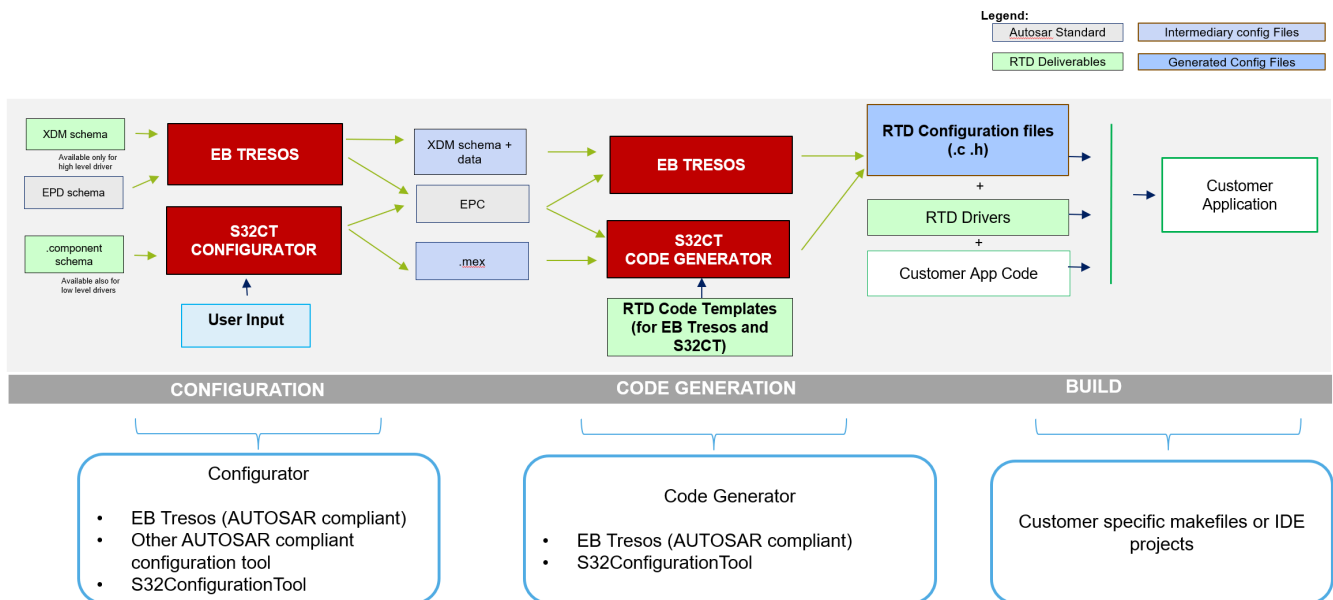


Figure 4.1 Configuration generation flow

- [Ocu](#)
- [FTM_OCUC](#)

4.1 Ocu

- Module [Ocu](#)
 - Container [OcuConfigurationOfOptionalApis](#)
 - * Parameter [OcuDeInitApi](#)
 - * Parameter [OcuGetCounterApi](#)
 - * Parameter [OcuNotificationSupported](#)
 - * Parameter [OcuSetAbsoluteThresholdApi](#)
 - * Parameter [OcuSetPinActionApi](#)
 - * Parameter [OcuSetPinStateApi](#)
 - * Parameter [OcuSetRelativeThresholdApi](#)
 - * Parameter [OcuVersionInfoApi](#)
 - Container [OcuConfigSet](#)
 - * Parameter [OcuCountdirection](#)
 - * Container [OcuChannel](#)
 - Parameter [OcuChannelId](#)
 - Parameter [OcuAssignedHardwareChannel](#)
 - Parameter [OcuDefaultThreshold](#)
 - Parameter [OcuHardwareTriggeredAdc](#)
 - Parameter [OcuHardwareTriggeredDMA](#)
 - Parameter [OcuMaxCounterValue](#)
 - Parameter [OcuNotification](#)
 - Parameter [OcuOutputPinUsed](#)
 - Parameter [OcuOutputPinDefaultState](#)
 - Parameter [OcuOutputPinAction](#)
 - Parameter [OcuUseMcuReferencePoint](#)
 - Parameter [OcuChannelTickDuration](#)
 - Reference [OcuChannelEcucPartitionRef](#)
 - Reference [OcuHWSpecificSettingsRef](#)
 - Reference [OcuMcuClockReferencePoint](#)
 - * Container [OcuHWSpecificSettings](#)
 - Parameter [OcuHardwareChannelId](#)
 - Parameter [OcuHardwareElements](#)
 - Parameter [OcuClockSource](#)
 - Parameter [OcuPrescale](#)
 - Parameter [OcuPrescale__Alternate](#)
 - Parameter [OcuDebugMode](#)

- Container [OcuGeneral](#)
 - * Parameter [OcuDevErrorDetect](#)
 - * Parameter [OcuEnableDualClockMode](#)
 - * Parameter [OcuEnableUserModeSupport](#)
 - * Parameter [OcuMultiPartitionEnabled](#)
 - * Reference [OcuEcucPartitionRef](#)
 - * Reference [OcuKernelEcucPartitionRef](#)
 - * Container [OcuHwResourceConfig](#)
 - Parameter [OcuHwResourceId](#)
 - Parameter [OcuIsrEnable](#)
 - Parameter [OcuChannelsUsed](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)

4.1.1 Module Ocu

Configuration of OCU (Output Compare Unit) module.

Included containers:

- [OcuConfigurationOfOptionalApis](#)
- [OcuConfigSet](#)
- [OcuGeneral](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.1.2 Container OcuConfigurationOfOptionalApis

Configuration of optional APIs.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Parameter OcuDeInitApi

Adds / removes the service Ocu_DeInit() from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.4 Parameter OcuGetCounterApi

Adds / removes the service Ocu_GetCounter() from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.5 Parameter OcuNotificationSupported

Adds / removes the services `Ocu_EnableNotification()` and `Ocu_DisableNotification()` from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.6 Parameter OcuSetAbsoluteThresholdApi

Adds / removes the service `Ocu_SetAbsoluteThreshold()` from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.7 Parameter OcuSetPinActionApi

Adds / removes the service `Ocu_SetPinAction()` from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.8 Parameter OcuSetPinStateApi

Adds / removes the service *Ocu_SetPinState()* from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.9 Parameter OcuSetRelativeThresholdApi

Adds / removes the service *Ocu_SetRelativeThreshold()* from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.10 Parameter OcuVersionInfoApi

Switch to indicate that the

Ocu_GetVersionInfo() is supported.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.11 Container OcuConfigSet

This container is the base of a Configuration Set, which contains the configured OCU channels. This way, different configuration sets can be defined for post-build process.

Included subcontainers:

- [OcuChannel](#)
- [OcuHWSpecificSettings](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.12 Parameter OcuCountdirection

This parameter indicates the count direction for the whole OCU driver.

Note This feature does not support down counting. Up-counting is always used as default

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OCU_UPCOUNTING
literals	['OCU_DOWNCOUNTING', 'OCU_UPCOUNTING']

4.1.13 Container OcuChannel

Configuration of an individual OCU channel.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.14 Parameter OcuChannelId

Channel Id of the OCU channel. This value will be assigned to the symbolic name derived from the OcuChannel container short name. It defines the assignment of the channel to the physical OCU hardware channel.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	65535
min	0

4.1.15 Parameter OcuAssignedHardwareChannel

The physical hardware channel that is assigned to this logical channel.

Note Hardware channel should be enumeration parameter and has been replaced by OcuHWSpecificSettingsRef

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	1
max	255
min	0

4.1.16 Parameter OcuDefaultThreshold

Value of comparison threshold used for Initialization.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	10000
max	65535
min	0

4.1.17 Parameter OcuHardwareTriggeredAdc

This parameter is used to allow the OCU channel to trigger an ADC channel upon compare match, if this is supported by hardware. The value of the parameter represents the ADC physical channel to trigger..

NoteTrigger for ADC should be routed via MCL and has not supported yet in OCU driver. Hardware trigger for ADC could be used by PWM

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0
max	255
min	0

4.1.18 Parameter OcuHardwareTriggeredDMA

This parameter is used to allow the OCU channel to trigger a DMA channel upon compare match, if this is supported by hardware. The value of the parameter represents the DMA physical channel to trigger.

NoteHardware trigger for DMA has not supported yet in OCU driver.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0
max	255
min	0

4.1.19 Parameter OcuMaxCounterValue

Maximum value in ticks, the counter of the OCU channel is able to count.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	65535
max	65535
min	1

4.1.20 Parameter OcuNotification

User callback function

NOTE: please use NULL or NULL_PTR w/o any quotes. If the used string is different from NULL or NULL_PTR it will be used as the configured function name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL

4.1.21 Parameter OcuOutputPinUsed

Information about the usage of an output pin on this channel.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.22 Parameter OcuOutputPinDefaultState

The parameter *OcuOutputPinDefaultState* represents the state that a pin associated with a channel shall be set to after initialization.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	OCU_LOW
literals	['OCU_HIGH', 'OCU_LOW']

4.1.23 Parameter OcuOutputPinAction

The parameter *OcuOutputPinAction* represents the action that a pin associated with a channel shall be set immediately after a compare match event.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	OCU_DISABLE
literals	['OCU_SET_LOW', 'OCU_SET_HIGH', 'OCU_TOGGLE', 'OCU_DISABLE']

4.1.24 Parameter OcuUseMcuReferencePoint

This parameter allows for automatic calculation of the Channel Tick duration based on the configured MCU Referenced Clock.

If *OcuUseMcuReferencePoint* is set to true then selecting a Clock source from MCU is possible and based on that source clock *OcuChannelTickDuration* can be auto-calculated.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.25 Parameter OcuChannelTickDuration

Specifies the tick duration of the counter of the channel. This parameter is the number of the input clock edges (rising edges or falling edges exclusively) counted each time to increase the counter by one unit.

Note. The default value is calculated using a default prescaler of 1. For a different prescaler value the calculated value should be divided by the said prescaler value.

Note Unit the OCU Channel Tick Duration is nano seconds(ns).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	1
max	32768
min	1

4.1.26 Reference OcuChannelEcucPartitionRef

Maps a OCU channel to zero or multiple ECUC partitions to limit the access to this channel group.

The ECUC partitions referenced are a subset of the ECUC partitions where the OCU driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0

Property	Value
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.27 Reference OcuHWSpecificSettingsRef

Reference to the OcuHWSpecificSettings used by the OcuChannel.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/Ocu/OcuConfigSet/OcuHWSpecificSettings

4.1.28 Reference OcuMcuClockReferencePoint

Reference to the Ftm clock source configuration,

which is set in the MCU driver configuration.

Note This parameter is not supported in current implementation.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

Configuration Parameters

Property	Value
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/Mcu/McuModuleConfiguration/McuClockSetting↔ Config/McuClockReferencePoint

4.1.29 Container OcuHWSpecificSettings

This container contains Ocu-specific parameters for selecting the clock source and setting optional prescalers if supported by hardware.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.30 Parameter OcuHardwareChannelId

Channel ID of the OCU hardware channel. This ID is used to match with OcuChannel/OcuAssignedHardwareChannel

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	65535
min	0

4.1.31 Parameter OcuHardwareElements

Selects one Ftm channel

available on the platform.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	FTM_0_CH_0
literals	['FTM_0_CH_0', 'FTM_0_CH_1', 'FTM_0_CH_2', 'FTM_0_CH_3', 'FTM_0_CH_4', 'FTM_0_CH_5', 'FTM_0_CH_6', 'FTM_0_CH_7', 'FTM_1_CH_0', 'FTM_1_CH_1', 'FTM_1_CH_2', 'FTM_1_CH_3', 'FTM_1_CH_4', 'FTM_1_CH_5', 'FTM_1_CH_6', 'FTM_1_CH_7', 'FTM_2_CH_0', 'FTM_2_CH_1', 'FTM_2_CH_2', 'FTM_2_CH_3', 'FTM_2_CH_4', 'FTM_2_CH_5', 'FTM_2_CH_6', 'FTM_2_CH_7', 'FTM_3_CH_0', 'FTM_3_CH_1', 'FTM_3_CH_2', 'FTM_3_CH_3', 'FTM_3_CH_4', 'FTM_3_CH_5', 'FTM_3_CH_6', 'FTM_3_CH_7']

4.1.32 Parameter OcuClockSource

The OCU driver specific clock input for the unit can statically be configured to select different clock sources if provided by hardware. Enumeration literals are defined vendor specific.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OCU_SYSTEM_CLOCK
literals	['OCU_SYSTEM_CLOCK', 'OCU_EXTERNAL_CLOCK']

4.1.33 Parameter OcuPrescale

Optional OCU driver specific clock prescale factor, if supported by hardware. Implementation is defined vendor specific.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF

Configuration Parameters

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	DIV1
literals	['DIV1', 'DIV2', 'DIV4', 'DIV8', 'DIV16', 'DIV32', 'DIV64', 'DIV128']

4.1.34 Parameter OcuPrescale__Alternate

Select input count source (clock) used for this Ftm module. This parameter will be used by the Ocu_SetClockMode function.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	DIV1
literals	['DIV1', 'DIV2', 'DIV4', 'DIV8', 'DIV16', 'DIV32', 'DIV64', 'DIV128']

4.1.35 Parameter OcuDebugMode

OcuDebugMode

These bits allow select the behavior of the FTM counter, the CH(n)F bit, the channels output, and the writing to the MOD, CNTIN, and C(n)V registers (wave form register)

CNT_ON_OUTPUTS_ON - Continue with normal operation during debug mode.

CNT_STOPED_OUTPUTS_FROZEN - Counter is stopped, channels outputs are frozen, writes to wave form registers bypass the registers buffers

CNT_STOPED_OUTPUTS_SAFE - Counter is stopped, channels outputs are forced to their safe value according to POLn bit, writes to wave form registers bypass the registers buffers

CNT_STOPED_FLAG_SET - Counter is stopped, channels outputs operate with its functionality, writes to wave form registers bypass the registers buffers

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	CNT_ON_OUTPUTS_ON
literals	['CNT_ON_OUTPUTS_ON', 'CNT_STOPED_OUTPUTS_FROZEN', 'CNT_STOPED_OUTPUTS_SAFE', 'CNT_STOPED_FLAG_SET']

4.1.36 Container OcuGeneral

This container contains the module-wide configuration parameters of the OCU Driver.

Included subcontainers:

- [OcuHwResourceConfig](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.37 Parameter OcuDevErrorDetect

Switch for enabling the development error detection.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.38 Parameter OcuEnableDualClockMode

Adds / removes the services `Ocu_SetClockMode()` from the code. This function is called when the prescaler value needs to be change to maintain same period at different frequency.

Note:

Note This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.39 Parameter OcuEnableUserModeSupport

When this parameter is enabled, the OCU module will adapt to run from User Mode, with the following measures:

Configuring `REG_PROT` for Ftm IP so that the registers under protection can be accessed from user mode by setting UAA bit in `REG_PROT_GCR` to 1

For more information, please see chapter 5.7 User Mode Support in IM

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.40 Parameter OcuMultiPartitionEnabled

Switch to enable/disable multi-partition feature.

User can choose `ENABLE` multi-partition feature by checking this option, this will force to configure at least 1 ECUC partition in `OcuChannelEcucPartitionRef`, and each OCU channel in `OcuChannel` to configure at least 1 ECUC partition reference

in *OcuChannelEcucPartitionRef* container to fulfill generating code condition; OR unchecked this option to *DISABLE*

multi-partition feature, performing this action will force user to remove all ECUC partition reference in every OCU channels contained

in *OcuChannel* and in *OcuChannelEcucPartitionRef*.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.41 Reference OcuEcucPartitionRef

Maps the OCU driver to zero or multiple ECUC partitions to make the driver API available in the according partition.

Depending on the addressed timer resource the interfaces operate as follows.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.42 Reference OcuKernelEcucPartitionRef

Maps the OCU kernel to zero or one ECUC partitions to assign the driver kernel to a certain partition.

The ECUC partition referenced is a subset of the ECUC partitions where the OCU driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC

Configuration Parameters

Property	Value
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.43 Container OcuHwResourceConfig

List of HW interrupts available for the entire platform.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	32
upperMultiplicity	32
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.44 Parameter OcuHwResourceId

Id of the HW interrupt service routine available platform wide and usable by the Ocu module.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	FTM_0_CH_1

Property	Value
literals	['FTM_0_CH_0', 'FTM_0_CH_1', 'FTM_0_CH_2', 'FTM_0_CH_3', 'FTM_0_CH_4', 'FTM_0_CH_5', 'FTM_0_CH_6', 'FTM_0_CH_7', 'FTM_1_CH_0', 'FTM_1_CH_1', 'FTM_1_CH_2', 'FTM_1_CH_3', 'FTM_1_CH_4', 'FTM_1_CH_5', 'FTM_1_CH_6', 'FTM_1_CH_7', 'FTM_2_CH_0', 'FTM_2_CH_1', 'FTM_2_CH_2', 'FTM_2_CH_3', 'FTM_2_CH_4', 'FTM_2_CH_5', 'FTM_2_CH_6', 'FTM_2_CH_7', 'FTM_3_CH_0', 'FTM_3_CH_1', 'FTM_3_CH_2', 'FTM_3_CH_3', 'FTM_3_CH_4', 'FTM_3_CH_5', 'FTM_3_CH_6', 'FTM_3_CH_7']

4.1.45 Parameter OcuIsrEnable

Status of the HW Interrupt (true - Interrupt shall be enable platform wide; false - Interrupt shall be disabled platform wide).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.46 Parameter OcuChannelsUsed

This column configures HW channels which are going to be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.47 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.48 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.1.49 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION

Property	Value
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	7
max	7
min	7

4.1.50 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.51 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	125
max	125
min	125

4.1.52 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	3
max	3
min	3

4.1.53 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.54 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.55 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

`<ModuleName>_<VendorId>_<VendorApiInfix>`.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name `Can_Write` defined in the SWS will translate to `Can_123_v11r456Write`.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.1.56 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

4.2 FTM_OCU

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Ftm_Ocu](#)
 - Container [OcuConfigurationOfOptionalApis](#)
 - * Parameter [OcuDeInitApi](#)
 - * Parameter [OcuGetCounterApi](#)
 - * Parameter [OcuNotificationSupported](#)
 - * Parameter [OcuSetAbsoluteThresholdApi](#)
 - * Parameter [OcuSetPinActionApi](#)
 - * Parameter [OcuSetPinStateApi](#)
 - * Parameter [OcuSetRelativeThresholdApi](#)
 - * Parameter [OcuVersionInfoApi](#)
 - Container [OcuConfigSet](#)
 - * Parameter [OcuCountdirection](#)
 - * Container [OcuChannel](#)
 - Parameter [OcuChannelId](#)
 - Parameter [OcuAssignedHardwareChannel](#)
 - Parameter [OcuDefaultThreshold](#)
 - Parameter [OcuHardwareTriggeredAdc](#)
 - Parameter [OcuHardwareTriggeredDMA](#)
 - Parameter [OcuMaxCounterValue](#)
 - Parameter [OcuNotification](#)
 - Parameter [OcuOutputPinUsed](#)
 - Parameter [OcuOutputPinDefaultState](#)
 - Parameter [OcuOutputPinAction](#)
 - Parameter [OcuUseMcuReferencePoint](#)
 - Parameter [OcuChannelTickDuration](#)

- Reference [OcuHWSpecificSettingsRef](#)
- Reference [OcuChannelEcucPartitionRef](#)
- Reference [OcuMcuClockReferencePoint](#)
- * Container [OcuHWSpecificSettings](#)
 - Parameter [OcuHardwareChannelId](#)
 - Parameter [OcuHardwareElements](#)
 - Parameter [OcuClockSource](#)
 - Parameter [OcuPrescale](#)
 - Parameter [OcuPrescale_Alternate](#)
 - Parameter [OcuDebugMode](#)
- Container [OcuGeneral](#)
 - * Parameter [OcuMultiPartitionEnabled](#)
 - * Parameter [OcuDevErrorDetect](#)
 - * Parameter [OcuEnableDualClockMode](#)
 - * Parameter [OcuEnableUserModeSupport](#)
 - * Parameter [OcuDownCountingSupport](#)
 - * Reference [OcuEcucPartitionRef](#)
 - * Reference [OcuKernelEcucPartitionRef](#)
 - * Container [OcuHwResourceConfig](#)
 - Parameter [OcuHwResourceId](#)
 - Parameter [OcuIsrEnable](#)
 - Parameter [OcuChannelsUsed](#)

4.2.1 Module Ftm_Ocu

Configuration of the Ftm_Ocu (Output Compare Unit) module.

Included containers:

- [OcuConfigurationOfOptionalApis](#)
- [OcuConfigSet](#)
- [OcuGeneral](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.2.2 Parameter OcuDownCountingSupport

When this parameter is enabled, it is possible to configure OcuCountdirection with OCU_DOWNCOUNTING.

Note eMios IP does not support down counting

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Ocu IPL	42
Ocu Driver	59

Chapter 6

Module Documentation

6.1 Ocu IPL

6.1.1 Detailed Description

Ocu FTM driver header file.

Ocu FTM driver header file specific to OCU driver.

Ocu FTM header file support for OCU driver.

FTM driver interface.

FTM specific defines which need to be exported to external application

Data Structures

- struct [Ftm_Ocu_Ip_ChStateType](#)
internal context structure [More...](#)
- struct [Ftm_Ocu_Ip_ChannelConfigType](#)
FTM IP channel specific configuration structure for the Ocu functionality. [More...](#)
- struct [Ftm_Ocu_Ip_SpecificIpConfigType](#)
FTM IP module specific configuration structure for the Ocu functionality. [More...](#)
- struct [Ftm_Ocu_Ip_ModuleConfigType](#)
FTM IP specific configuration structure type. [More...](#)

Macros

- `#define FTM_OCU_IP_COMBINE_CH_CTRL_WIDTH`
Combine control width for channel combine function.
- `#define FTM_OCU_IP_INVALID_CHANNEL_NUM`
invalid FTM channel number
- `#define FTM_OCU_IP_BDM_MODE_00`
Options for the FlexTimer behavior in BDM Mode.
- `#define FTM_OCU_IP_BDM_MODE_01`
- `#define FTM_OCU_IP_BDM_MODE_10`
- `#define FTM_OCU_IP_BDM_MODE_11`

- #define `FTM_OCU_IP_CLOCK_SOURCE_NONE`
FlexTimer Clock Source Selection.
- #define `FTM_OCU_IP_CLOCK_SOURCE_SYSTEMCLK`
- #define `FTM_OCU_IP_CLOCK_SOURCE_FIXEDCLK`
- #define `FTM_OCU_IP_CLOCK_SOURCE_EXTERNALCLK`
- #define `FTM_OCU_IP_CLOCK_DIVID_BY_1`
FlexTimer Prescale Factor Selections bits (PS).
- #define `FTM_OCU_IP_CLOCK_DIVID_BY_2`
- #define `FTM_OCU_IP_CLOCK_DIVID_BY_4`
- #define `FTM_OCU_IP_CLOCK_DIVID_BY_8`
- #define `FTM_OCU_IP_CLOCK_DIVID_BY_16`
- #define `FTM_OCU_IP_CLOCK_DIVID_BY_32`
- #define `FTM_OCU_IP_CLOCK_DIVID_BY_64`
- #define `FTM_OCU_IP_CLOCK_DIVID_BY_128`
- #define `FTM_OCU_IP_SET_HIGH`
Edge Pin Action values.
- #define `FTM_OCU_IP_SET_LOW`
The channel pin will be set HIGH upon compare match.
- #define `FTM_OCU_IP_SET_TOGGLE`
The channel pin will be set to the opposite of its current level HIGH upon compare match.
- #define `FTM_OCU_IP_SET_DISABLE`
The channel pin will remain at its current level upon compare match.
- #define `FTM_OCU_IP_HIGH`
Pin State levels.
- #define `FTM_OCU_IP_LOW`
Ocu Pin level is logic low.
- #define `FTM_OCU_IP_PRIMARY_PRESCALER`
Prescaler values.
- #define `FTM_OCU_IP_ALTERNATIVE_PRESCALER`
Selected value is the alternative configured prescaler.
- #define `FTM_OCU_IP_CM_IN_REF_INTERVAL`
Ocu Return values.
- #define `FTM_OCU_IP_CM_OUT_REF_INTERVAL`
The compare match will not occur inside the current Reference Interval.

Types Reference

- typedef uint8 `Ftm_Ocu_Ip_PinActionType`
Edge Pin Action type.
- typedef uint8 `Ftm_Ocu_Ip_PinStateType`
Pin State level.
- typedef uint8 `Ftm_Ocu_Ip_SelectPrescalerType`

Prescaler type.

- typedef uint8 [Ftm_Ocu_Ip_ReturnType](#)

Ocu Return Type.

- typedef FTM_Type [Ftm_Ocu_Ip_xRegLayoutType](#)
- typedef void(* [Ftm_Ocu_Ip_CallbackType](#)) (uint16 CallbackParam)

Channel notification typedef.

- typedef uint16 [Ftm_Ocu_Ip_ValueType](#)

FTM Value type (the value of the period is platform dependent and thus configurable)

Enum Reference

- enum [Ftm_Ocu_Ip_OutputCompareModeType](#)

FlexTimer output compare edge mode. Toggle, clear or set.

Function Reference

- void [Ftm_Ocu_Ip_Init](#) (const [Ftm_Ocu_Ip_ModuleConfigType](#) *const pFtmIpConfig)

This function initializes all internals variables of the driver.

- void [Ftm_Ocu_Ip_DeInit](#) (uint8 InstNum)

This function deinitializes the driver.

- void [Ftm_Ocu_Ip_StartChannel](#) (uint8 InstNum, uint8 ChNum)

Ocu driver function for starting the FTM timer channel.

- void [Ftm_Ocu_Ip_StopChannel](#) (uint8 InstNum, uint8 ChNum)

Ocu driver function for stopping the FTM timer channel.

- [Ftm_Ocu_Ip_ValueType](#) [Ftm_Ocu_Ip_GetCounter](#) (uint8 InstNum)

Ocu driver function for getting the current counter value for a FTM timer channel.

- void [Ftm_Ocu_Ip_SetPinAction](#) (uint8 InstNum, uint8 ChNum, [Ftm_Ocu_Ip_PinActionType](#) PinAction)

Ocu driver function for setting the Pin Action for the current FTM channel.

- void [Ftm_Ocu_Ip_SetPinState](#) (uint8 InstNum, uint8 ChNum, [Ftm_Ocu_Ip_PinStateType](#) PinState)

Ocu driver function for setting the Pin State for the current FTM channel.

- [Ftm_Ocu_Ip_ReturnType](#) [Ftm_Ocu_Ip_SetRelativeThreshold](#) (uint8 InstNum, uint8 ChNum, [Ftm_Ocu_Ip_ValueType](#) RelativeValue)

The function sets the value of the channel threshold relative to the current value of the counter.

- [Ftm_Ocu_Ip_ReturnType](#) [Ftm_Ocu_Ip_SetAbsoluteThreshold](#) (uint8 InstNum, uint8 ChNum, [Ftm_Ocu_Ip_ValueType](#) ReferenceValue, [Ftm_Ocu_Ip_ValueType](#) AbsoluteValue)

The function set the value of the channel threshold using an absolute input data.

- void [Ftm_Ocu_Ip_SetClockMode](#) (uint8 InstNum, [Ftm_Ocu_Ip_SelectPrescalerType](#) Prescaler)

The function changes the prescaler of FTM channel.

- uint16 [Ftm_Ocu_Ip_GetMaxCounterValue](#) (uint8 InstNum, uint8 ChNum)

Ftm_Ocu_Ip_GetMaxCounterValue.

- void [Ftm_Ocu_Ip_DisableNotification](#) (uint8 InstNum, uint8 ChNum)

This function is used to disable the OCU compare match notification.

- void [Ftm_Ocu_Ip_EnableNotification](#) (uint8 InstNum, uint8 ChNum)

This function is used to enable the OCU compare match notification of the indexed channel.

- INTERRUPT_FUNC void [FTM_0_CH_0_CH_1_ISR](#) (void)

Independent interrupt handler.

- INTERRUPT_FUNC void [FTM_0_CH_2_CH_3_ISR](#) (void)

Independent interrupt handler.

- INTERRUPT_FUNC void [FTM_0_CH_4_CH_5_ISR](#) (void)

Independent interrupt handler.

- void [Ftm_Ocu_Ip_SetOutputPinCtrlBit](#) (uint8 InstNum)

Set OBE CTRL bit in SIM module to set all channel pin of respective instance is output mode.

- #define [FTM_OCU_IP_OUTPUTDISABLED](#)

FTM register define used to configure counter clock source.

6.1.2 Data Structure Documentation

6.1.2.1 struct Ftm_Ocu_Ip_ChStateType

internal context structure

This structure is used by the IPL driver for internal logic. The content is populated on Init

Definition at line 314 of file Ftm_Ocu_Ip_Types.h.

6.1.2.2 struct Ftm_Ocu_Ip_ChannelConfigType

FTM IP channel specific configuration structure for the Ocu functionality.

Definition at line 324 of file Ftm_Ocu_Ip_Types.h.

Data Fields

const uint8	HwChannel	Assigned FTM channel Id.
const uint16	DefaultThreshold	Compare match threshold for the current channel.
const uint8	ChannelControlValue	FTM channel parameters. Bits 7 --> 1: Output Pin is Used / 0: Output Pin is not used Bit 6 --> Default Pin State: 0: PIN_LOW; 1: PIN_HIGH Bits 5 .. 4 --> Pin Action behaviour on compare match: SET_PIN_LOW; 2: SET_PIN_HIGH; 1: PIN_TOGGLE; 0: OUTPUT_DISABLE Bits 3 .. 0 --> Reserved for future use
Ftm_Ocu_Ip_CallbackType	mCallbackFunc	Channel notification function.
uint16	mCallbackParam	Channel notification parameter.

6.1.2.3 struct Ftm_Ocu_Ip_SpecificIpConfigType

FTM IP module specific configuration structure for the Ocu functionality.

Definition at line 345 of file Ftm_Ocu_Ip_Types.h.

Data Fields

const uint16	MaxCounterValue	FTM maximum counter value.
--------------	-----------------	----------------------------

Data Fields

const uint8	ModuleControlValue	FTM module parameters. Bits 7 .. 6 --> Clock source Bits 5 .. 3 --> Normal prescale Bit 2 .. 1 --> Debug mode config Bits 0 --> Reserved for future use
const uint8	AlternateCtrVal	FTM channel parameters. Bits 7 .. 5 --> Alternate Prescaler configuration Bits 4 .. 0 --> Reserved for future use

6.1.2.4 struct Ftm_Ocu_Ip_ModuleConfigType

FTM IP specific configuration structure type.

Definition at line 364 of file Ftm_Ocu_Ip_Types.h.

Data Fields

const uint8	ModuleId	Number of this FTM module.
const uint8	NumChannels	Number of FTM channels configured for this FTM module.
const Ftm_Ocu_Ip_ChannelConfigType (*	pcxChannelsConfig)[]	Pointer to the channels configuration structure.
const Ftm_Ocu_Ip_SpecificIpConfigType *	pcxModulesConfig	Pointer to the common module configuration structure.

6.1.3 Macro Definition Documentation

6.1.3.1 FTM_OCU_IP_COMBINE_CH_CTRL_WIDTH

```
#define FTM_OCU_IP_COMBINE_CH_CTRL_WIDTH
```

Combine control width for channel combine function.

Definition at line 88 of file Ftm_Ocu_Ip_Types.h.

6.1.3.2 FTM_OCU_IP_INVALID_CHANNEL_NUM

```
#define FTM_OCU_IP_INVALID_CHANNEL_NUM
```

invalid FTM channel number

Definition at line 91 of file Ftm_Ocu_Ip_Types.h.

6.1.3.3 FTM_OCU_IP_OUTPUTDISABLED

```
#define FTM_OCU_IP_OUTPUTDISABLED
```

FTM register define used to configure counter clock source.

Definition at line 97 of file Ftm_Ocu_Ip_Types.h.

6.1.3.4 FTM_OCU_IP_BDM_MODE_00

```
#define FTM_OCU_IP_BDM_MODE_00
```

Options for the FlexTimer behavior in BDM Mode.

FTM counter stopped, CH(n)F bit can be set, FTM channels in functional mode, writes to MOD, CNTIN and C(n)V registers bypass the register buffers

Definition at line 158 of file Ftm_Ocu_Ip_Types.h.

6.1.3.5 FTM_OCU_IP_BDM_MODE_01

```
#define FTM_OCU_IP_BDM_MODE_01
```

FTM counter stopped, CH(n)F bit is not set, FTM channels outputs are forced to their safe value, writes to MOD, CNTIN and C(n)V registers bypass the register buffers

Definition at line 161 of file Ftm_Ocu_Ip_Types.h.

6.1.3.6 FTM_OCU_IP_BDM_MODE_10

```
#define FTM_OCU_IP_BDM_MODE_10
```

FTM counter stopped, CH(n)F bit is not set, FTM channels outputs are frozen when chip enters in BDM mode, writes to MOD, CNTIN and C(n)V registers bypass the register buffers

Definition at line 164 of file Ftm_Ocu_Ip_Types.h.

6.1.3.7 FTM_OCU_IP_BDM_MODE_11

```
#define FTM_OCU_IP_BDM_MODE_11
```

FTM counter in functional mode, CH(n)F bit can be set, FTM channels in functional mode, writes to MOD, CNTIN and C(n)V registers is in fully functional mode

Definition at line 167 of file Ftm_Ocu_Ip_Types.h.

6.1.3.8 FTM_OCU_IP_CLOCK_SOURCE_NONE

```
#define FTM_OCU_IP_CLOCK_SOURCE_NONE
```

FlexTimer Clock Source Selection.

None use clock for FTM

Definition at line 173 of file Ftm_Ocu_Ip_Types.h.

6.1.3.9 FTM_OCU_IP_CLOCK_SOURCE_SYSTEMCLK

```
#define FTM_OCU_IP_CLOCK_SOURCE_SYSTEMCLK
```

System clock

Definition at line 174 of file Ftm_Ocu_Ip_Types.h.

6.1.3.10 FTM_OCU_IP_CLOCK_SOURCE_FIXEDCLK

```
#define FTM_OCU_IP_CLOCK_SOURCE_FIXEDCLK
```

Fixed clock

Definition at line 175 of file Ftm_Ocu_Ip_Types.h.

6.1.3.11 FTM_OCU_IP_CLOCK_SOURCE_EXTERNALCLK

```
#define FTM_OCU_IP_CLOCK_SOURCE_EXTERNALCLK
```

External clock

Definition at line 176 of file Ftm_Ocu_Ip_Types.h.

6.1.3.12 FTM_OCU_IP_CLOCK_DIVID_BY_1

```
#define FTM_OCU_IP_CLOCK_DIVID_BY_1
```

FlexTimer Prescale Factor Selections bits (PS).

Selects one of 8 division factors for the clock source selected by CLKS. The new prescaler factor affects the clock source on the next FTM input clock cycle after the new value is updated into the register bits. This field is write protected. It can be written only when MODE[WPDIS] = 1. 000b - Divide by 1 001b - Divide by 2 010b - Divide by 4 011b - Divide by 8 100b - Divide by 16 101b - Divide by 32 110b - Divide by 64 111b - Divide by 128 Divide by 1

Definition at line 194 of file Ftm_Ocu_Ip_Types.h.

6.1.3.13 FTM_OCU_IP_CLOCK_DIVID_BY_2

```
#define FTM_OCU_IP_CLOCK_DIVID_BY_2
```

Divide by 2

Definition at line 195 of file Ftm_Ocu_Ip_Types.h.

6.1.3.14 FTM_OCU_IP_CLOCK_DIVID_BY_4

```
#define FTM_OCU_IP_CLOCK_DIVID_BY_4
```

Divide by 4

Definition at line 196 of file Ftm_Ocu_Ip_Types.h.

6.1.3.15 FTM_OCU_IP_CLOCK_DIVID_BY_8

```
#define FTM_OCU_IP_CLOCK_DIVID_BY_8
```

Divide by 8

Definition at line 197 of file Ftm_Ocu_Ip_Types.h.

6.1.3.16 FTM_OCU_IP_CLOCK_DIVID_BY_16

```
#define FTM_OCU_IP_CLOCK_DIVID_BY_16
```

Divide by 16

Definition at line 198 of file Ftm_Ocu_Ip_Types.h.

6.1.3.17 FTM_OCU_IP_CLOCK_DIVID_BY_32

```
#define FTM_OCU_IP_CLOCK_DIVID_BY_32
```

Divide by 32

Definition at line 199 of file Ftm_Ocu_Ip_Types.h.

6.1.3.18 FTM_OCU_IP_CLOCK_DIVID_BY_64

```
#define FTM_OCU_IP_CLOCK_DIVID_BY_64
```

Divide by 64

Definition at line 200 of file Ftm_Ocu_Ip_Types.h.

6.1.3.19 FTM_OCU_IP_CLOCK_DIVID_BY_128

```
#define FTM_OCU_IP_CLOCK_DIVID_BY_128
```

Divide by 128

Definition at line 201 of file Ftm_Ocu_Ip_Types.h.

6.1.3.20 FTM_OCU_IP_SET_HIGH

```
#define FTM_OCU_IP_SET_HIGH
```

Edge Pin Action values.

Automatic action (by hardware) to be performed on a pin attached to an OCU channel.

The channel pin will be set LOW upon compare match.

Definition at line 209 of file Ftm_Ocu_Ip_Types.h.

6.1.3.21 FTM_OCU_IP_SET_LOW

```
#define FTM_OCU_IP_SET_LOW
```

The channel pin will be set HIGH upon compare match.

Definition at line 211 of file Ftm_Ocu_Ip_Types.h.

6.1.3.22 FTM_OCU_IP_SET_TOGGLE

```
#define FTM_OCU_IP_SET_TOGGLE
```

The channel pin will be set to the opposite of its current level HIGH upon compare match.

Definition at line 213 of file Ftm_Ocu_Ip_Types.h.

6.1.3.23 FTM_OCU_IP_SET_DISABLE

```
#define FTM_OCU_IP_SET_DISABLE
```

The channel pin will remain at its current level upon compare match.

Definition at line 215 of file Ftm_Ocu_Ip_Types.h.

6.1.3.24 FTM_OCU_IP_HIGH

```
#define FTM_OCU_IP_HIGH
```

Pin State levels.

Output state of the pin linked to an OCU channel.

Ocu Pin level is logic high

Definition at line 224 of file Ftm_Ocu_Ip_Types.h.

6.1.3.25 FTM_OCU_IP_LOW

```
#define FTM_OCU_IP_LOW
```

Ocu Pin level is logic low.

Definition at line 226 of file Ftm_Ocu_Ip_Types.h.

6.1.3.26 FTM_OCU_IP_PRIMARY_PRESCALER

```
#define FTM_OCU_IP_PRIMARY_PRESCALER
```

Prescaler values.

Possible values of prescalers used to configure base-clock timers

Selected value is the default/primary prescaler

Definition at line 234 of file Ftm_Ocu_Ip_Types.h.

6.1.3.27 FTM_OCU_IP_ALTERNATIVE_PRESCALER

```
#define FTM_OCU_IP_ALTERNATIVE_PRESCALER
```

Selected value is the alternative configured prescaler.

Definition at line 236 of file Ftm_Ocu_Ip_Types.h.

6.1.3.28 FTM_OCU_IP_CM_IN_REF_INTERVAL

```
#define FTM_OCU_IP_CM_IN_REF_INTERVAL
```

Ocu Return values.

Return information after setting a new threshold value.

The compare match will occur inside the current Reference Interval.

Definition at line 244 of file Ftm_Ocu_Ip_Types.h.

6.1.3.29 FTM_OCU_IP_CM_OUT_REF_INTERVAL

```
#define FTM_OCU_IP_CM_OUT_REF_INTERVAL
```

The compare match will not occur inside the current Reference Interval.

Definition at line 246 of file Ftm_Ocu_Ip_Types.h.

6.1.4 Types Reference**6.1.4.1 Ftm_Ocu_Ip_PinActionType**

```
typedef uint8 Ftm_Ocu_Ip_PinActionType
```

Edge Pin Action type.

Automatic action (by hardware) to be performed on a pin attached to an OCU channel.

Definition at line 269 of file Ftm_Ocu_Ip_Types.h.

6.1.4.2 Ftm_Ocu_Ip_PinStateType

```
typedef uint8 Ftm_Ocu_Ip_PinStateType
```

Pin State level.

Output state of the pin linked to an OCU channel.

Definition at line 276 of file Ftm_Ocu_Ip_Types.h.

6.1.4.3 Ftm_Ocu_Ip_SelectPrescalerType

```
typedef uint8 Ftm_Ocu_Ip_SelectPrescalerType
```

Prescaler type.

Specifies the possible values of prescalers used to configure base-clock timers

Definition at line 283 of file Ftm_Ocu_Ip_Types.h.

6.1.4.4 Ftm_Ocu_Ip_ReturnType

```
typedef uint8 Ftm_Ocu_Ip_ReturnType
```

Ocu Return Type.

Return information after setting a new threshold value.

Definition at line 290 of file Ftm_Ocu_Ip_Types.h.

6.1.4.5 Ftm_Ocu_Ip_xRegLayoutType

```
typedef FTM_Type Ftm_Ocu_Ip_xRegLayoutType
```

Redefine FTM Register Layout Typedef from header file to comply with coding guidelines

Definition at line 293 of file Ftm_Ocu_Ip_Types.h.

6.1.4.6 Ftm_Ocu_Ip_CallbackType

```
typedef void(* Ftm_Ocu_Ip_CallbackType) (uint16 CallbackParam)
```

Channel notification typedef.

Definition at line 296 of file Ftm_Ocu_Ip_Types.h.

6.1.4.7 Ftm_Ocu_Ip_ValueType

```
typedef uint16 Ftm_Ocu_Ip_ValueType
```

FTM Value type (the value of the period is platform dependent and thus configurable)

Type for reading the counter and writing the threshold values (in number of ticks)

Definition at line 304 of file Ftm_Ocu_Ip_Types.h.

6.1.5 Enum Reference

6.1.5.1 Ftm_Ocu_Ip_OutputCompareModeType

enum `Ftm_Ocu_Ip_OutputCompareModeType`

FlexTimer output compare edge mode. Toggle, clear or set.

Definition at line 253 of file `Ftm_Ocu_Ip_Types.h`.

6.1.6 Function Reference

6.1.6.1 Ftm_Ocu_Ip_Init()

```
void Ftm_Ocu_Ip_Init (
    const Ftm_Ocu_Ip_ModuleConfigType *const pFtmIpConfig )
```

This function initializes all internal variables of the driver.

This function will initialize with default values all FTM registers. Will loop through all FTM modules and channels in the configuration structure and will setup required registers for each channel and module

Parameters

in	<code>pFtmIpConfig</code>	- Pointer to FTM IP configuration structure
----	---------------------------	---

6.1.6.2 Ftm_Ocu_Ip_DeInit()

```
void Ftm_Ocu_Ip_DeInit (
    uint8 InstNum )
```

This function deinitializes the driver.

This function will set to idle state all FTM channels. Will stop all FTM modules. Disables all FTM channels interrupts.

Parameters

in	<code>InstNum</code>	- FTM instance number
----	----------------------	-----------------------

6.1.6.3 Ftm_Ocu_Ip_StartChannel()

```
void Ftm_Ocu_Ip_StartChannel (
    uint8 InstNum,
    uint8 ChNum )
```

Ocu driver function for starting the FTM timer channel.

This function: Configure ELSP:ELSA Release software control Clear flag and enable interrupt

Parameters

in	<i>InstNum</i>	- FTM instance number
in	<i>ChNum</i>	- Channel number in the FTM instance

6.1.6.4 Ftm_Ocu_Ip_StopChannel()

```
void Ftm_Ocu_Ip_StopChannel (
    uint8 InstNum,
    uint8 ChNum )
```

Ocu driver function for stopping the FTM timer channel.

This function:

- stop the timer channel by writing the ELSB:ELSA = 0:0
- Clear flag and disable interrupt

Parameters

in	<i>InstNum</i>	- FTM instance number
in	<i>ChNum</i>	- Channel number in the FTM instance

6.1.6.5 Ftm_Ocu_Ip_GetCounter()

```
Ftm_Ocu_Ip_ValueType Ftm_Ocu_Ip_GetCounter (
    uint8 InstNum )
```

Ocu driver function for getting the current counter value for a FTM timer channel.

This function is called for reading the FTM channel counter register.

Parameters

in	<i>InstNum</i>	- FTM instance number
----	----------------	-----------------------

return register CNTR current value

6.1.6.6 Ftm_Ocu_Ip_SetPinAction()

```
void Ftm_Ocu_Ip_SetPinAction (
    uint8 InstNum,
    uint8 ChNum,
    Ftm_Ocu_Ip_PinActionType PinAction )
```

Ocu driver function for setting the Pin Action for the current FTM channel.

Parameters

in	<i>InstNum</i>	- FTM instance number
----	----------------	-----------------------

Parameters

in	<i>ChNum</i>	- Channel number in the FTM instance PinAction - Type of action: FTM_OCU_IP_SET_LOW, FTM_OCU_IP_SET_HIGH, FTM_OCU_IP_SET_TOGGLE
----	--------------	---

6.1.6.7 Ftm_Ocu_Ip_SetPinState()

```
void Ftm_Ocu_Ip_SetPinState (
    uint8 InstNum,
    uint8 ChNum,
    Ftm_Ocu_Ip_PinStateType PinState )
```

Ocu driver function for setting the Pin State for the current FTM channel.

Parameters

in	<i>InstNum</i>	- FTM instance number
in	<i>ChNum</i>	- Channel number in the FTM instance PinState - Expectation state of pin: FTM_OCU_IP_HIGH, FTM_OCU_IP_LOW

6.1.6.8 Ftm_Ocu_Ip_SetRelativeThreshold()

```
Ftm_Ocu_Ip_ReturnType Ftm_Ocu_Ip_SetRelativeThreshold (
    uint8 InstNum,
    uint8 ChNum,
    Ftm_Ocu_Ip_ValueType RelativeValue )
```

The function sets the value of the channel threshold relative to the current value of the counter.

On entry, the API reads the counter value (ReadValue). Then the new threshold value is computed and written according to the following formula: $\text{NewThresholdValue} = \text{ReadValue} + \text{RelativeValue}$ or $\text{NewThresholdValue} = (\text{ReadValue} + \text{RelativeValue}) - \text{MaxCounterValue}$ if matching occur at next cycle

Parameters

in	<i>InstNum</i>	- FTM instance number
in	<i>ChNum</i>	- Channel number in the FTM instance RelativeValue - Channel timeout value

return Tells the caller whether the compare match will occur (or has already occurred) during the current Reference Interval, as a result of setting the new threshold value.

6.1.6.9 Ftm_Ocu_Ip_SetAbsoluteThreshold()

```
Ftm_Ocu_Ip_ReturnType Ftm_Ocu_Ip_SetAbsoluteThreshold (
    uint8 InstNum,
    uint8 ChNum,
    Ftm_Ocu_Ip_ValueType ReferenceValue,
    Ftm_Ocu_Ip_ValueType AbsoluteValue )
```

The function set the value of the channel threshold using an absolute input data.

Parameters

in	<i>InstNum</i>	- FTM instance number
in	<i>ChNum</i>	- Channel number in the FTM instance
in	<i>ReferenceValue</i>	- Value given by the upper layer and used as a base to determine whether to call the notification before the function exits or not.
in	<i>AbsoluteValue</i>	- channel timeout value

return Tells the caller whether the compare match will occur (or has already occurred) during the current Reference Interval, as a result of setting the new threshold value.

6.1.6.10 Ftm_Ocu_Ip_SetClockMode()

```
void Ftm_Ocu_Ip_SetClockMode (
    uint8 InstNum,
    Ftm_Ocu_Ip_SelectPrescalerType Prescaler )
```

The function changes the prescaler of FTM channel.

This function changes the value prescaler in SC registers

Parameters

in	<i>Prescaler</i>	- Prescaler type: normal or alternative
in	<i>InstNum</i>	- FTM instance number

6.1.6.11 Ftm_Ocu_Ip_GetMaxCounterValue()

```
uint16 Ftm_Ocu_Ip_GetMaxCounterValue (
    uint8 InstNum,
    uint8 ChNum )
```

Ftm_Ocu_Ip_GetMaxCounterValue.

This function will return max counter value for given channel

Parameters

in	<i>InstNum</i>	- FTM instance number
in	<i>ChNum</i>	- Channel number in the FTM instance

Returns

uint16 - Max counter value for given channel

6.1.6.12 Ftm_Ocu_Ip_DisableNotification()

```
void Ftm_Ocu_Ip_DisableNotification (
    uint8 InstNum,
```

```
uint8 ChNum )
```

This function is used to disable the OCU compare match notification.

Parameters

in	<i>InstNum</i>	- FTM instance number
in	<i>ChNum</i>	- Channel number in the FTM instance

Returns

none

6.1.6.13 Ftm_Ocu_Ip_EnableNotification()

```
void Ftm_Ocu_Ip_EnableNotification (
    uint8 InstNum,
    uint8 ChNum )
```

This function is used to enable the OCU compare match notification of the indexed channel.

Parameters

in	<i>InstNum</i>	- FTM instance number
in	<i>ChNum</i>	- Channel number in the FTM instance

Returns

none

6.1.6.14 FTM_0_CH_0_CH_1_ISR()

```
INTERRUPT_FUNC void FTM_0_CH_0_CH_1_ISR (
    void )
```

Independent interrupt handler.

Interrupt handler for FTM module 0 channel 0 - channel 1.

6.1.6.15 FTM_0_CH_2_CH_3_ISR()

```
INTERRUPT_FUNC void FTM_0_CH_2_CH_3_ISR (
    void )
```

Independent interrupt handler.

Interrupt handler for FTM module 0 channel 2 - channel 3.

6.1.6.16 FTM_0_CH_4_CH_5_ISR()

```
INTERRUPT_FUNC void FTM_0_CH_4_CH_5_ISR (
    void )
```

Independent interrupt handler.

Interrupt handler for FTM module 0 channel 4 - channel 5.

6.1.6.17 Ftm_Ocu_Ip_SetOutputPinCtrlBit()

```
void Ftm_Ocu_Ip_SetOutputPinCtrlBit (
    uint8 InstNum )
```

Set OBE CTRL bit in SIM module to set all channel pin of respective instance is output mode.

Parameters

in	<i>The</i>	FTM peripheral Instance number
----	------------	--------------------------------

6.2 Ocu Driver

6.2.1 Detailed Description

Ocu HLD module API header.

Ocu HLD generic types file.

Ocu HLD notifications API header.

Ocu HLD Environment configuration header file.

Ocu module API header, containing Autosar API and function that are exported by OCU driver

Ocu environment configuration header.

Ocu notifications API header.

OCU header file containing generic Ocu types and structures.

Data Structures

- struct [Ocu_ChannelConfigType](#)
Ocu high level channel configuration structure. [More...](#)
- struct [Ocu_ConfigType](#)
Ocu high level configuration structure. [More...](#)

Macros

- `#define OCU_VALID_CHANNEL_NUM`
Valid channel number.
- `#define OCU_INVALID_CHANNEL_NUM`
Invalid channel number.
- `#define OCU_INSTANCE_INDEX`
Instance ID of this OCU driver.
- `#define OCU_E_UNINIT`
API service used without module initialization.
- `#define OCU_E_PARAM_INVALID_CHANNEL`
API service used with an invalid channel Identifier.
- `#define OCU_E_PARAM_INVALID_STATE`
API [Ocu_SetPinState\(\)](#) called with an invalid pin state or when the channel is in the RUNNING state..
- `#define OCU_E_PARAM_INVALID_ACTION`
API [Ocu_SetPinAction\(\)](#) called with an invalid pin action.
- `#define OCU_E_NO_VALID_NOTIF`
Usage of [Ocu_DisableNotification\(\)](#) or [Ocu_EnableNotification\(\)](#) on a channel where a NULL pointer is configured as the notification function.
- `#define OCU_E_ALREADY_INITIALIZED`
API [Ocu_Init\(\)](#) called while the OCU driver has already been initialized.
- `#define OCU_E_PARAM_POINTER`
API [Ocu_GetVersionInfo\(\)](#) is called with a NULL parameter.

- `#define OCU_E_BUSY`
API `Ocu_StartChannel()` called on a channel that is in state `RUNNING`.
- `#define OCU_E_PARAM_NO_PIN`
`Ocu_SetPinState()` or `Ocu_SetPinAction()` called for a channel that doesn't have an associated output pin.
- `#define OCU_E_INIT_FAILED`
API `Ocu_Init` service called with wrong parameter.
- `#define OCU_E_PARAM_INVALID_VALUE`
`Ocu_SetAbsoluteThreshold()` or `Ocu_SetRelativeThreshold()` called for with a compare match parameter greater than maximum supported counter value for a given channel.
- `#define OCU_E_PARAM_CONFIG`
API `Ocu_Init` service called with wrong Core number.
- `#define OCU_INIT_ID`
API service ID of `Ocu_Init()` function.
- `#define OCU_DEINIT_ID`
API service ID of `Ocu_DeInit()` function.
- `#define OCU_STARTCHANNEL_ID`
API service ID of `Ocu_StartChannel()` function.
- `#define OCU_STOPCHANNEL_ID`
API service ID of `Ocu_StopChannel()` function.
- `#define OCU_SETPINSTATE_ID`
API service ID of `Ocu_SetPinState()` function.
- `#define OCU_SETPINACTION_ID`
API service ID of `Ocu_SetPinAction()` function.
- `#define OCU_GETCOUNTER_ID`
API service ID of `Ocu_GetCounter()` function.
- `#define OCU_SETABSOLUTETHRESHOLD_ID`
API service ID of `Ocu_SetAbsoluteThreshold()` function.
- `#define OCU_SETRELATIVETHRESHOLD_ID`
API service ID of `Ocu_SetRelativeThreshold()` function.
- `#define OCU_GETVERSIONINFO_ID`
API service ID of `Ocu_GetVersionInfo()` function.
- `#define OCU_DISABLENOTIFICATION_ID`
API service ID of `Ocu_DisableNotification()` function.
- `#define OCU_ENABLENOTIFICATION_ID`
API service ID of `Ocu_EnableNotification()` function.
- `#define OCU_SETCLOCKMODE_ID`
API `Ocu_SetClockMode` service called with wrong parameter.

Types Reference

- typedef void(* [Ocu_NotificationType](#)) (void)
Ocu channel notification typedef.
- typedef uint32 [Ocu_ValueType](#)
Ocu Value type (the value of the period is platform dependent and thus configurable)
- typedef uint16 [Ocu_ChannelType](#)
Ocu channel type.

Enum Reference

- enum [Ocu_ReturnType](#)
Ocu Return Type.
- enum [Ocu_PinActionType](#)
Edge Pin Action type.
- enum [Ocu_PinStateType](#)
Pin State level.
- enum [Ocu_SelectPrescalerType](#)
Prescaler type.

Function Reference

- void [Ocu_Init](#) (const [Ocu_ConfigType](#) *ConfigPtr)
This function initializes the Ocu driver.
- void [Ocu_DeInit](#) (void)
This function deinitializes the Ocu driver.
- Std_ReturnType [Ocu_StartChannel](#) ([Ocu_ChannelType](#) ChannelNumber)
This function starts a specified Ocu channel.
- void [Ocu_StopChannel](#) ([Ocu_ChannelType](#) ChannelNumber)
This function stops a specified Ocu channel.
- void [Ocu_SetPinState](#) ([Ocu_ChannelType](#) ChannelNumber, [Ocu_PinStateType](#) PinState)
Service to set immediately the level of the pin associated to an OCU channel.
- void [Ocu_SetPinAction](#) ([Ocu_ChannelType](#) ChannelNumber, [Ocu_PinActionType](#) PinAction)
Service to indicate the driver what shall be done automatically by hardware (if supported) upon compare match.
- [Ocu_ValueType](#) [Ocu_GetCounter](#) ([Ocu_ChannelType](#) ChannelNumber)
Service to read the current value of the counter.
- [Ocu_ReturnType](#) [Ocu_SetAbsoluteThreshold](#) ([Ocu_ChannelType](#) ChannelNumber, [Ocu_ValueType](#) ReferenceValue, [Ocu_ValueType](#) AbsoluteValue)
Service to set the value of the channel threshold using an absolute input data.
- [Ocu_ReturnType](#) [Ocu_SetRelativeThreshold](#) ([Ocu_ChannelType](#) ChannelNumber, [Ocu_ValueType](#) RelativeValue)
Service to set the value of the channel threshold relative to the current value of the counter.
- void [Ocu_DisableNotification](#) ([Ocu_ChannelType](#) ChannelNumber)
This service is used to disable notifications from an OCU channel.

- void [Ocu_EnableNotification](#) ([Ocu_ChannelType](#) ChannelNumber)
This service is used to enable notifications from an OCU channel.
- void [Ocu_GetVersionInfo](#) ([Std_VersionInfoType](#) *versioninfo)
This function returns Ocu driver version details.
- void [Ocu_SetClockMode](#) ([Ocu_SelectPrescalerType](#) Prescaler)
Implementation specific function to change the peripheral clock frequency.
- void [Ocu_ProcessChannelNotification](#) (uint16 Channel)
Ocu common handler to implements generic part of the ISR.

Variables

- const [Ocu_ConfigType](#) [Ocu_Config_VS_0](#)
Export Post-Build Ocu configurations.

6.2.2 Data Structure Documentation

6.2.2.1 struct [Ocu_ChannelConfigType](#)

Ocu high level channel configuration structure.

Definition at line 387 of file [Ocu_Types.h](#).

Data Fields

boolean	OutputPinEnable	Channel output pin enable.
const Ocu_NotificationType	pfChNotification	Pointer to channel notification function.

6.2.2.2 struct [Ocu_ConfigType](#)

Ocu high level configuration structure.

Definition at line 401 of file [Ocu_Types.h](#).

Data Fields

- const [Ocu_ChannelType](#) [NumChannels](#)
Number of OCU channels (configured in tresos plugin builder)
- const [Ocu_ChannelConfigType](#)(* [pOcuChannelsConfig](#))[]
Pointer to the OCU channel configuration.
- const [Ocu_Ipw_IpConfigType](#) * [pcxIpConfig](#)
Combined IP specific configuration structure.
- const [Ocu_ChannelType](#)(* [HwToLogicChannelMap](#))[]
Index table to translate HW channels to logical used to process interrupts for notifications.
- uint8 [PartitionId](#)
Partition Index.

Field Documentation

NumChannels

```
const Ocu_ChannelType NumChannels
```

Number of OCU channels (configured in tresos plugin builder)

Definition at line 404 of file Ocu_Types.h.

pOcuChannelsConfig

```
const Ocu_ChannelConfigType(* pOcuChannelsConfig) [ ]
```

Pointer to the OCU channel configuration.

Definition at line 406 of file Ocu_Types.h.

pcxIpConfig

```
const Ocu_Ipw_IpConfigType* pcxIpConfig
```

Combined IP specific configuration structure.

Definition at line 408 of file Ocu_Types.h.

HwToLogicChannelMap

```
const Ocu_ChannelType(* HwToLogicChannelMap) [ ]
```

Index table to translate HW channels to logical used to process interrupts for notifications.

Definition at line 411 of file Ocu_Types.h.

PartitionId

```
uint8 PartitionId
```

Partition Index.

Definition at line 415 of file Ocu_Types.h.

6.2.3 Macro Definition Documentation**6.2.3.1 OCU_VALID_CHANNEL_NUM**

```
#define OCU_VALID_CHANNEL_NUM
```

Valid channel number.

Definition at line 120 of file Ocu_Types.h.

6.2.3.2 OCU_INVALID_CHANNEL_NUM

```
#define OCU_INVALID_CHANNEL_NUM
```

Invalid channel number.

Definition at line 123 of file Ocu_Types.h.

6.2.3.3 OCU_INSTANCE_INDEX

```
#define OCU_INSTANCE_INDEX
```

Instance ID of this OCU driver.

Definition at line 126 of file Ocu_Types.h.

6.2.3.4 OCU_E_UNINIT

```
#define OCU_E_UNINIT
```

API service used without module initialization.

Errors and exceptions that will be detected by the OCU driver AUTOSAR

Definition at line 134 of file Ocu_Types.h.

6.2.3.5 OCU_E_PARAM_INVALID_CHANNEL

```
#define OCU_E_PARAM_INVALID_CHANNEL
```

API service used with an invalid channel Identifier.

Errors and exceptions that will be detected by the OCU driver AUTOSAR

Definition at line 141 of file Ocu_Types.h.

6.2.3.6 OCU_E_PARAM_INVALID_STATE

```
#define OCU_E_PARAM_INVALID_STATE
```

API [Ocu_SetPinState\(\)](#) called with an invalid pin state or when the channel is in the RUNNING state..

Errors and exceptions that will be detected by the OCU driver AUTOSAR

Definition at line 148 of file Ocu_Types.h.

6.2.3.7 OCU_E_PARAM_INVALID_ACTION

```
#define OCU_E_PARAM_INVALID_ACTION
```

API [Ocu_SetPinAction\(\)](#) called with an invalid pin action.

Errors and exceptions that will be detected by the OCU driver AUTOSAR

Definition at line 155 of file Ocu_Types.h.

6.2.3.8 OCU_E_NO_VALID_NOTIF

```
#define OCU_E_NO_VALID_NOTIF
```

Usage of [Ocu_DisableNotification\(\)](#) or [Ocu_EnableNotification\(\)](#) on a channel where a NULL pointer is configured as the notification function.

Errors and exceptions that will be detected by the OCU driver. AUTOSAR

Definition at line 163 of file Ocu_Types.h.

6.2.3.9 OCU_E_ALREADY_INITIALIZED

```
#define OCU_E_ALREADY_INITIALIZED
```

API [Ocu_Init\(\)](#) called while the OCU driver has already been initialized.

Errors and exceptions that will be detected by the OCU driver. AUTOSAR

Definition at line 170 of file Ocu_Types.h.

6.2.3.10 OCU_E_PARAM_POINTER

```
#define OCU_E_PARAM_POINTER
```

API [Ocu_GetVersionInfo\(\)](#) is called with a NULL parameter.

Errors and exceptions that will be detected by the OCU driver. AUTOSAR

Definition at line 177 of file Ocu_Types.h.

6.2.3.11 OCU_E_BUSY

```
#define OCU_E_BUSY
```

API [Ocu_StartChannel\(\)](#) called on a channel that is in state RUNNING.

Errors and exceptions that will be detected by the OCU driver AUTOSAR

Definition at line 184 of file Ocu_Types.h.

6.2.3.12 OCU_E_PARAM_NO_PIN

```
#define OCU_E_PARAM_NO_PIN
```

[Ocu_SetPinState\(\)](#) or [Ocu_SetPinAction\(\)](#) called for a channel that doesn't have an associated output pin.

Errors and exceptions that will be detected by the OCU driver AUTOSAR

Definition at line 192 of file Ocu_Types.h.

6.2.3.13 OCU_E_INIT_FAILED

```
#define OCU_E_INIT_FAILED
```

API Ocu_Init service called with wrong parameter.

Errors and exceptions that will be detected by the OCU driver AUTOSAR

Definition at line 200 of file Ocu_Types.h.

6.2.3.14 OCU_E_PARAM_INVALID_VALUE

```
#define OCU_E_PARAM_INVALID_VALUE
```

[Ocu_SetAbsoluteThreshold\(\)](#) or [Ocu_SetRelativeThreshold\(\)](#) called for with a compare match parameter greater than maximum supported counter value for a given channel.

Errors and exceptions that will be detected by the OCU driver Non-AUTOSAR

Definition at line 208 of file Ocu_Types.h.

6.2.3.15 OCU_E_PARAM_CONFIG

```
#define OCU_E_PARAM_CONFIG
```

API Ocu_Init service called with wrong Core number.

Errors and exceptions that will be detected by the OCU driver when Non-AUTOSAR

Definition at line 215 of file Ocu_Types.h.

6.2.3.16 OCU_INIT_ID

```
#define OCU_INIT_ID
```

API service ID of [Ocu_Init\(\)](#) function.

Parameters used when raising an error/exception

Definition at line 221 of file Ocu_Types.h.

6.2.3.17 OCU_DEINIT_ID

```
#define OCU_DEINIT_ID
```

API service ID of [Ocu_DeInit\(\)](#) function.

Parameters used when raising an error/exception

Definition at line 227 of file Ocu_Types.h.

6.2.3.18 OCU_STARTCHANNEL_ID

```
#define OCU_STARTCHANNEL_ID
```

API service ID of [Ocu_StartChannel\(\)](#) function.

Parameters used when raising an error/exception

Definition at line 233 of file Ocu_Types.h.

6.2.3.19 OCU_STOPCHANNEL_ID

```
#define OCU_STOPCHANNEL_ID
```

API service ID of [Ocu_StopChannel\(\)](#) function.

Parameters used when raising an error/exception

Definition at line 239 of file Ocu_Types.h.

6.2.3.20 OCU_SETPINSTATE_ID

```
#define OCU_SETPINSTATE_ID
```

API service ID of [Ocu_SetPinState\(\)](#) function.

Parameters used when raising an error/exception

Definition at line 244 of file Ocu_Types.h.

6.2.3.21 OCU_SETPINACTION_ID

```
#define OCU_SETPINACTION_ID
```

API service ID of [Ocu_SetPinAction\(\)](#) function.

Parameters used when raising an error/exception

Definition at line 250 of file Ocu_Types.h.

6.2.3.22 OCU_GETCOUNTER_ID

```
#define OCU_GETCOUNTER_ID
```

API service ID of [Ocu_GetCounter\(\)](#) function.

Parameters used when raising an error/exception

Definition at line 256 of file Ocu_Types.h.

6.2.3.23 OCU_SETABSOLUTETHRESHOLD_ID

```
#define OCU_SETABSOLUTETHRESHOLD_ID
```

API service ID of [Ocu_SetAbsoluteThreshold\(\)](#) function.

Parameters used when raising an error/exception

Definition at line 262 of file Ocu_Types.h.

6.2.3.24 OCU_SETRELATIVETHRESHOLD_ID

```
#define OCU_SETRELATIVETHRESHOLD_ID
```

API service ID of [Ocu_SetRelativeThreshold\(\)](#) function.

Parameters used when raising an error/exception

Definition at line 268 of file Ocu_Types.h.

6.2.3.25 OCU_GETVERSIONINFO_ID

```
#define OCU_GETVERSIONINFO_ID
```

API service ID of [Ocu_GetVersionInfo\(\)](#) function.

Parameters used when raising an error/exception

Definition at line 274 of file Ocu_Types.h.

6.2.3.26 OCU_DISABLENOTIFICATION_ID

```
#define OCU_DISABLENOTIFICATION_ID
```

API service ID of [Ocu_DisableNotification\(\)](#) function.

Parameters used when raising an error/exception

Definition at line 280 of file Ocu_Types.h.

6.2.3.27 OCU_ENABLENOTIFICATION_ID

```
#define OCU_ENABLENOTIFICATION_ID
```

API service ID of [Ocu_EnableNotification\(\)](#) function.

Parameters used when raising an error/exception

Definition at line 286 of file Ocu_Types.h.

6.2.3.28 OCU_SETCLOCKMODE_ID

```
#define OCU_SETCLOCKMODE_ID
```

API Ocu_SetClockMode service called with wrong parameter.

Parameters used when raising an error/exception

Definition at line 292 of file Ocu_Types.h.

6.2.4 Types Reference

6.2.4.1 Ocu_NotificationType

```
typedef void(* Ocu_NotificationType) (void)
```

Ocu channel notification typedef.

The callback notifications shall be configurable as pointers to user defined functions within the configuration structure.

Definition at line 307 of file Ocu_Types.h.

6.2.4.2 Ocu_ValueType

```
typedef uint32 Ocu_ValueType
```

Ocu Value type (the value of the period is platform dependent and thus configurable)

Type for reading the counter and writing the threshold values (in number of ticks)

Definition at line 314 of file Ocu_Types.h.

6.2.4.3 Ocu_ChannelType

```
typedef uint16 Ocu_ChannelType
```

Ocu channel type.

Numeric identifier of an OCU channel.

Definition at line 320 of file Ocu_Types.h.

6.2.5 Enum Reference

6.2.5.1 Ocu_ReturnType

```
enum Ocu_ReturnType
```

Ocu Return Type.

Return information after setting a new threshold value.

Enumerator

OCU_CM_IN_REF_INTERVAL	The compare match will occur inside the current Reference Interval.
OCU_CM_OUT_REF_INTERVAL	The compare match will not occur inside the current Reference Interval.

Definition at line 329 of file Ocu_Types.h.

6.2.5.2 Ocu_PinActionType

enum `Ocu_PinActionType`

Edge Pin Action type.

Automatic action (by hardware) to be performed on a pin attached to an OCU channel.

Enumerator

OCU_SET_HIGH	The channel pin will be set HIGH upon compare match.
OCU_SET_LOW	The channel pin will be set LOW upon compare match.
OCU_TOGGLE	The channel pin will be set to the opposite of its current level HIGH upon compare match.
OCU_DISABLE	The channel pin will remain at its current level upon compare match.

Definition at line 342 of file Ocu_Types.h.

6.2.5.3 Ocu_PinStateType

enum `Ocu_PinStateType`

Pin State level.

Output state of the pin linked to an OCU channel.

Enumerator

OCU_HIGH	The pin associated to an OCU channel is in high state.
OCU_LOW	The pin associated to an OCU channel is in low state.

Definition at line 361 of file Ocu_Types.h.

6.2.5.4 Ocu_SelectPrescalerType

enum `Ocu_SelectPrescalerType`

Prescaler type.

This enum specifies the possible types of prescallers used to configure base-clock timers

Enumerator

OCU_PRIMARY_PRESCALER	Selected value is the default/primary prescaler.
OCU_ALTERNATIVE_PRESCALER	Selected value is the alternative configured prescaler.

Definition at line 374 of file Ocu_Types.h.

6.2.6 Function Reference

6.2.6.1 Ocu_Init()

```
void Ocu_Init (
    const Ocu_ConfigType * ConfigPtr )
```

This function initializes the Ocu driver.

The function Ocu_Init shall initialize all internal variables and the used OCU structure of the microcontroller according to the parameters specified in ConfigPtr. Ocu shall initialize the free-running timers that are used by the driver.

If development error detection is enabled, calling the routine with a NULL ConfigPtr, Ocu_Init shall raise the development error OCU_E_INIT_FAILED and return without any action.

If development error detection is enabled, calling the routine Ocu_Init while the OCU driver and hardware are already initialized will cause a development error OCU_E_ALREADY_INITIALIZED. The desired functionality shall be left without any action.

For pre-compile and link time configuration variants, a NULL pointer shall be passed to the initialization routine. In this case the check for this NULL pointer has to be omitted.

If development error detection for the Ocu module is enabled, if any function (except Ocu_Init) is called before Ocu_Init has been called, the called function shall raise development error OCU_E_UNINIT.

Parameters

in	ConfigPtr	pointer to OCU top configuration structure
----	-----------	--

Returns

void

6.2.6.2 Ocu_DeInit()

```
void Ocu_DeInit (
    void )
```

This function deinitializes the Ocu driver.

The function Ocu_DeInit shall deinitialize the OCU module.

The function Ocu_DeInit shall deinitialize the OCU variables and registers that were initialized by Ocu_Init to a state comparable to their power on reset state.

The function Ocu_DeInit shall disable OCU interrupts and OCU signal edge notifications.

The function Ocu_DeInit shall stop all free-running counters, which are exclusively used by this driver.

If development error detection for the Ocu module is enabled, when a development error occurs, the corresponding OCU function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).

If development error detection is enabled for the OCU driver: if a channel is still in the RUNNING state when the function Ocu_DeInit is called, then the function shall raise the development error OCU_E_PARAM_INVALID_STATE and return without any action.

If development error detection for the Ocu module is enabled, if any function (except Ocu_Init) is called before Ocu_DeInit has been called, the called function shall raise development error OCU_E_UNINIT.

Returns

void

Precondition

Ocu_Init

6.2.6.3 Ocu_StartChannel()

```
Std_ReturnType Ocu_StartChannel (
    Ocu_ChannelType ChannelNumber )
```

This function starts a specified Ocu channel.

The function Ocu_StartChannel shall start an OCU channel by allowing all compare match configured actions to be performed.

The state of the selected channel shall be set to RUNNING If the function Ocu_StartChannel has been successfully performed.

If development error detection is enabled for the OCU driver: If the function Ocu_StartChannel is called on a channel in the state RUNNING, then the function shall raise the error OCU_E_BUSY and return without any action.

If development error detection is enabled for the OCU driver: If the parameter ChannelNumber is invalid (not within the range specified by the configuration), the function Ocu_StartChannel shall raise the error OCU_E_PARAM←_INVALID_CHANNEL and return without any action.

If development error detection is enabled for the OCU driver: If the driver is not initialized, the function Ocu_←StartChannel shall raise the error OCU_E_UNINIT and return without any action.

Parameters

in	<i>ChannelNumber</i>	Ocu channel Id
----	----------------------	----------------

Returns

E_NOT_OK if call to Ocu_ValidateChannelConfigCall() or Ocu_ValidateRunningState() has failed, E_OK otherwise.

Precondition

Ocu_Init

6.2.6.4 Ocu_StopChannel()

```
void Ocu_StopChannel (
    Ocu_ChannelType ChannelNumber )
```

This function stops a specified Ocu channel.

The function Ocu_StopChannel shall stop an OCU channel by halting compare match configured actions for this channel.

The state of the selected channel shall be set to STOPPED if the function Ocu_StopChannel is successfully performed. If the function Ocu_StopChannel is called on a channel in the state STOPPED, then the function shall leave without any action (no change of the channel state), and shall not raise a development error.

If development error detection is enabled for the OCU driver: If the parameter `ChannelNumber` is invalid (not within the range specified by the configuration), the function `Ocu_StopChannel` shall raise the error `OCU_E_PARAM_INVALID_CHANNEL` and return without any action.

If development error detection is enabled for the OCU driver: If the driver is not initialized, the function `Ocu_StopChannel` shall raise the error `OCU_E_UNINIT` and return without any action.

Parameters

in	<i>ChannelNumber</i>	- Ocu channel Id
----	----------------------	------------------

Returns

void

Precondition

`Ocu_Init`

6.2.6.5 Ocu_SetPinState()

```
void Ocu_SetPinState (
    Ocu_ChannelType ChannelNumber,
    Ocu_PinStateType PinState )
```

Service to set immediately the level of the pin associated to an OCU channel.

The function `Ocu_SetPinState` shall set the pin associated with the channel to the level indicated by `PinState`.

The function `Ocu_SetPinState` shall be used only if the channel is not in the `RUNNING` state.

If development error detection is enabled for the OCU driver: If the parameter `ChannelNumber` is invalid (not within the range specified by the configuration), the function `Ocu_SetPinState` shall raise the error `OCU_E_PARAM_INVALID_CHANNEL` and return without any action.

If development error detection is enabled for the OCU driver: If a pin is not associated with the channel (not defined in the configuration of the channel), the function `Ocu_SetPinState` shall raise the error `OCU_E_PARAM_NO_PIN` and return without any action.

If development error detection is enabled for the OCU driver: If the parameter `PinState` is invalid (not within the range specified by the configuration), the function `Ocu_SetPinState` shall raise the error `OCU_E_PARAM_INVALID_STATE` and return without any action.

If development error detection is enabled for the OCU driver: If the driver is not initialized, the function `Ocu_SetPinState` shall raise the error `OCU_E_UNINIT` and return without any action.

If development error detection is enabled for the OCU driver: If the channel is in the `RUNNING` state, the function `Ocu_SetPinState` shall raise the error `OCU_E_PARAM_INVALID_STATE` and return without any action.

Parameters

in	<i>ChannelNumber</i>	- Ocu channel Id
in	<i>PinState</i>	- Output Pin State

Returns

void

Precondition

Ocu_Init

6.2.6.6 Ocu_SetPinAction()

```
void Ocu_SetPinAction (
    Ocu_ChannelType ChannelNumber,
    Ocu_PinActionType PinAction )
```

Service to indicate the driver what shall be done automatically by hardware (if supported) upon compare match.

The function Ocu_SetPinAction shall set the action to be performed by hardware automatically, at the next compare match in the corresponding OCU channel.

If development error detection is enabled for the OCU driver: If the parameter ChannelNumber is invalid (not within the range specified by the configuration), the function Ocu_SetPinAction shall raise the error OCU_E_PARAM_←_INVALID_CHANNEL and return without any action.

If development error detection is enabled for the OCU driver: If a pin is not associated with the channel (not defined in the configuration of the channel), the function Ocu_SetPinAction shall raise the error OCU_E_PARAM_NO_PIN and return without any action.

If development error detection is enabled for the OCU driver: If the parameter PinAction is invalid (not within the range specified by the type), the function Ocu_SetPinAction shall raise the error OCU_E_PARAM_INVALID_←_ACTION and return without any action.

If development error detection is enabled for the OCU driver: If the driver is not initialized, the function Ocu_Set_←PinAction shall raise the error OCU_E_UNINIT and return without any action.

If a pin is associated with a channel; the relevant action with this pin shall be performed upon compare match.

Parameters

in	ChannelNumber	- Ocu channel Id
in	PinAction	- Pin Action (OCU_SET_LOW, OCU_SET_HIGH, OCU_TOGGLE, OCU_DISABLE)

Returns

void

Precondition

Ocu_Init

6.2.6.7 Ocu_GetCounter()

```
Ocu_ValueType Ocu_GetCounter (
    Ocu_ChannelType ChannelNumber )
```

Service to read the current value of the counter.

The function Ocu_GetCounter shall read and return the value of the counter of the channel indicated by Channel_←Number.

If development error detection is enabled for the OCU driver: If the parameter ChannelNumber is invalid (not within the range specified by the configuration), the function Ocu_GetCounter shall raise the error OCU_E_PARAM_←INVALID_CHANNEL and shall return the value 0.

If development error detection is enabled for the OCU driver: if the driver is not initialized, then the function `Ocu_GetCounterValue` shall raise the error `OCU_E_UNINIT` and shall return the value 0.

Parameters

in	<i>ChannelNumber</i>	Ocu channel Id
----	----------------------	----------------

Returns

`Ocu_ValueType` the value of the hardware counter.

Precondition

`Ocu_Init`

6.2.6.8 Ocu_SetAbsoluteThreshold()

```
Ocu_ReturnType Ocu_SetAbsoluteThreshold (
    Ocu_ChannelType ChannelNumber,
    Ocu_ValueType ReferenceValue,
    Ocu_ValueType AbsoluteValue )
```

Service to set the value of the channel threshold using an absolute input data.

The function `Ocu_SetAbsoluteThreshold` shall set the channel threshold (the compare value) to the value given by `AbsoluteValue`.

If development error detection is enabled for the OCU driver: If the driver is not initialized, the function `Ocu_SetAbsoluteThreshold` shall raise the error `OCU_E_UNINIT` and return without any action.

If development error detection is enabled for the OCU driver: If the parameter `ChannelNumber` is invalid (not within the range specified by the configuration), the function `Ocu_SetAbsoluteThreshold` shall raise the error `OCU_E_PARAM_INVALID_CHANNEL` and return without any action.

After setting a new threshold value, the API `Ocu_SetAbsoluteThreshold` shall return a status to inform the caller whether the compare match will occur (or has already occurred) during the current Reference Interval, as a result of setting the new threshold value. (see `OCU_SWS` for details)

Parameters

in	<i>ChannelNumber</i>	- Ocu channel Id
in	<i>ReferenceValue</i>	- Value given by the upper layer and used as a base to determine whether to call the notification before the function exits or not.
in	<i>AbsoluteValue</i>	- Value to compare with the content of the counter. This value is in ticks.

Returns

`Ocu_ReturnType` - Tells the caller whether the compare match will occur (or has already occurred) during the current Reference Interval, as a result of setting the new threshold value.

Precondition

Ocu_Init

6.2.6.9 Ocu_SetRelativeThreshold()

```
Ocu_ReturnType Ocu_SetRelativeThreshold (
    Ocu_ChannelType ChannelNumber,
    Ocu_ValueType RelativeValue )
```

Service to set the value of the channel threshold relative to the current value of the counter.

The function Ocu_SetAbsoluteThreshold shall set the channel threshold (the compare value) to the value given by RelativeValue plus the current counter value read from hw.

After setting a new threshold value, the API Ocu_SetRelativeThreshold shall return a status to inform the caller whether the compare match will occur (or has already occurred) during the current Reference Interval, as a result of setting the new threshold value.

If development error detection is enabled for the OCU driver: If the driver is not initialized, the function Ocu_SetRelativeThreshold shall raise the error OCU_E_UNINIT and return without any action.

After setting a new threshold value, the API Ocu_SetRelativeThreshold shall return a status to inform the caller whether the compare match will occur (or has already occurred) during the current Reference Interval, as a result of setting the new threshold value.

Parameters

in	ChannelNumber	- Ocu channel Id
in	RelativeValue	- Value to use for computing the new threshold.

Returns

Ocu_ReturnType - Tells the caller whether the compare match will occur (or has already occurred) during the current Reference Interval, as a result of setting the new threshold value.

Precondition

Ocu_Init

6.2.6.10 Ocu_DisableNotification()

```
void Ocu_DisableNotification (
    Ocu_ChannelType ChannelNumber )
```

This service is used to disable notifications from an OCU channel.

The function Ocu_DisableNotification shall disable the OCU compare match notification.

If development error detection is enabled for the OCU driver: If the driver is not initialized, the function Ocu_DisableNotification shall raise the error OCU_E_UNINIT and return without any action.

If development error detection is enabled for the OCU driver: If the parameter Channel is invalid (not within the range specified by configuration), the function Ocu_DisableNotification shall raise the error OCU_E_PARAM_INVALID_CHANNEL and return without any action.

If development error detection is enabled for the OCU driver: If the notification function is the NULL pointer, the function Ocu_DisableNotification shall raise the error OCU_E_NO_VALID_NOTIF and return without any action.

Parameters

in	<i>ChannelNumber</i>	- Ocu channel Id
----	----------------------	------------------

Returns

void

Precondition

Ocu_Init

6.2.6.11 Ocu_EnableNotification()

```
void Ocu_EnableNotification (
    Ocu_ChannelType ChannelNumber )
```

This service is used to enable notifications from an OCU channel.

The function Ocu_EnableNotification shall enable the OCU compare match notification of the indexed channel.

If development error detection is enabled for the OCU driver: If the driver is not initialized, the function Ocu_↵ EnableNotification shall raise the error OCU_E_UNINIT and return without any action.

If development error detection is enabled for the OCU driver: If the parameter Channel is invalid (not within the range specified by configuration), then the function Ocu_EnableNotification shall raise the error OCU_E_PARA↵ M_INVALID_CHANNEL and return without any action.

If development error detection is enabled for the OCU driver: If the notification function is the NULL pointer, the function Ocu_EnableNotification shall raise the error OCU_E_NO_VALID_NOTIF and return without any action.

Parameters

in	<i>ChannelNumber</i>	- Ocu channel Id
in	<i>Notification</i>	- notification type to be enabled

Returns

void

Precondition

Ocu_Init

6.2.6.12 Ocu_GetVersionInfo()

```
void Ocu_GetVersionInfo (
    Std_VersionInfoType * versioninfo )
```

This function returns Ocu driver version details.

The function Ocu_GetVersionInfo shall return the version information of this module. The version information includes: Module Id, Vendor Id, Vendor specific version number.

Parameters

in, out	<i>versioninfo</i>	- pointer to Std_VersionInfoType output variable
---------	--------------------	--

Returns

void

6.2.6.13 Ocu_SetClockMode()

```
void Ocu_SetClockMode (
    Ocu_SelectPrescalerType Prescaler )
```

Implementation specific function to change the peripheral clock frequency.

This function is useful to set the prescalers that divide the OCU channels clock frequency.

Parameters

in	<i>Prescaler</i>	- prescaler type Possible values: • OCU_PRIMARY_PRESCALER for normal clock mode or • OCU_ALTERNATIVE_PRESCALER for a different prescaler for the driver
----	------------------	---

6.2.6.14 Ocu_ProcessChannelNotification()

```
void Ocu_ProcessChannelNotification (
    uint16 Channel )
```

Ocu common handler to implements generic part of the ISR.

Generic function used by all interrupt service routines to call notification

Parameters

in	<i>Channel</i>	- logic channel number
----	----------------	------------------------

6.2.7 Variable Documentation

6.2.7.1 Ocu_Config_VS_0

```
const Ocu_ConfigType Ocu_Config_VS_0 [extern]
```

Export Post-Build Ocu configurations.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.

